A Project Report On

Intelligent Human Face Deepfake Detection System with Explainable AI

Submitted in partial fulfillment of the requirements for the award of the degree of

MASTER OF COMPUTER APPLICATIONS

Submitted by

PANDI SUREKHA

Register Number: 248445100045

Under the esteemed guidance of

Mrs. P S V D GAYATRI

Assistant Professor

DEPARTMENT OF COMPUTER SCIENCE

ADIKAVI NANNAYA UNIVERSITY MSN CAMPUS KAKINADA

2024-2026

ADIKAVI NANNAYA UNIVERSITY MSN CAMPUS
KAKINADA

CERTIFICATE

This is to certify that the project entitled **"INTELLIGENT HUMAN FACE DEEPFAKE DETECTION SYSTEM WITH EXPLAINABLE AI"** is a Bonafide work carried out by **PANDI SUREKHA,** Roll No: **248445100045** in partial fulfillment of the requirements for the award of the degree of **Master of Computer Applications (MCA)** at **Adikavi Nannaya University MSN Campus, Kakinada**, under the guidance of **Mrs. P. S. V. D. GAYATRI**, **Department of Computer Science**, during the academic year 2025-2026.

PROJECT GUIDE **HEAD OF THE DEPARTMENT**

EXTERNAL EXAMINER

DECLARATION

I hereby declare that this project entitled **"INTELLIGENT HUMAN FACE DEEPFAKE DETECTION SYSTEM WITH EXPLAINABLE AI"** is an original work carried out by me.

a. The work contained in this report is original and has been done by me under the guidance of my supervisor(s).
b. This work has not been submitted to any other Institute for any degree or diploma.
c. I have followed the guidelines provided by the university in preparing the report.
d. I have conformed to the norms and guidelines given in the Ethical Code of Conduct of the Institute.
e. Whenever I have used materials (data, theoretical analysis, figures, and text) from other sources, I have given due credit to them by citing them in the text of the report and giving their details in the references. Further, I have taken permission from the copyright owners of the sources, whenever necessary.

Signature of the Student

Pandi Surekha (248445100045)

ACKNOWLEDGEMENT

I feel fortunate to pursue my **Master of Computer Applications** Degree in the Campus of **Department of Computer Science**, **Adikavi Nannaya University MSN Campus**, **Kakinada**. It provided all the facilities in the areas of Computer Science and Engineering.

It's a genuine pleasure to express my deep sense of thanks and gratitude to my mentor and project guide **Mrs. P S V D GAYATRI**, **Assistant Professor**, for her excellent guidance right from selection of project and her valuable suggestions throughout the project work. Her constant and timely counsel has been the key to my success, in completing this project in the college. She has given me a tremendous support in the technical front.

I profusely thank **Prof. S. Prasanthi Sri**, **Principal of Adikavi Nannaya University MSN Campus** for all the encouragement and support.

I also thank **Mr. Md. R. NADEEM**, **Assistant Professor**, and **Mr. K N S S V Prasad**, **Assistant Professor**, **Department of Computer Science** for the guidance, support and timely suggestions throughout the academic.

A great deal of thanks goes to the **Review Committee Members** and the **faculty members** of the **Department of Computer Science**, **Adikavi Nannaya University MSN Campus**, **Kakinada**, and the **Department of Computer Science and Engineering, University College of Engineering**, **Adikavi Nannaya University, Rajamahendravaram**, for their excellent guidance, supervision, and valuable suggestions.

I am always thankful to my family, friends and well-wishers for all their trust and constant support in the successful completion of my project.

PANDI SUREKHA
Reg.No:248445100045

ABSTRACT

The rapid advancement of artificial intelligence has enabled the creation of highly realistic manipulated videos known as deepfakes, which pose serious threats by spreading misinformation, influencing public opinion, and enabling digital fraud. Existing detection systems often function as black boxes, lacking transparency and reducing user trust. This project aims to develop an accurate and explainable deepfake detection system that not only identifies fake content but also provides clear insights into the reasoning behind its predictions.

The proposed **Intelligent Human Face Deepfake Detection System uses Explainable Artificial Intelligence (XAI)** to classify videos as real or fake. The system processes uploaded videos by extracting frames and detecting faces using OpenCV Haar Cascade classifiers, followed by preprocessing. A pretrained InceptionV3 model is used for feature extraction, generating 2048-dimensional feature vectors, which are then analyzed using a sequence model to capture temporal dependencies. Explainability techniques such as Grad-CAM and SHAP are integrated to highlight important regions and frame-level contributions. The system is implemented using TensorFlow, Keras, OpenCV, and Streamlit, providing accurate predictions along with visual explanations, thereby improving transparency, reliability, and trust in deepfake detection.

Keywords: Deepfake Detection, Computer Vision, Convolutional Neural Networks (CNN), InceptionV3, Explainable Artificial Intelligence (XAI), Grad-CAM, SHAP

INDEX

CHAPTER			PAGE NO
ABSTRACT			i
LIST OF FIGURES			iv-v
LIST OF TABLES			v
1.	**INTRODUCTION**		1-2
	1.1	Problem Definition	2
	1.2	Purpose of the System	2-3
	1.3	Scope of the System	3
	1.4	Motivation Behind the Problem Selection	3-4
	1.5	Existing System & Drawbacks	4-5
	1.6	Proposed System	5-6
2.	**REQUIREMENT ANALYSIS**		7
	2.1	Functional Requirements	7
	2.2	Non-Functional Requirements	8
	2.3	Software Requirements	8
	2.4	Hardware Requirements	8
3.	**SYSTEM ANALYSIS**		
	3.1	Introduction	9
	3.2	Literature Survey	9-11
	3.3	Feasibility Study	11
		3.3.1 Technical Feasibility	11
		3.3.2 Economical Feasibility	12
		3.3.3 Social Feasibility	12
4.	**SYSTEM DESIGN**		
	4.1	Introduction	13
	4.2	System Architecture	13-16
	4.3	System Methodology	16-18
	4.4	UML Diagrams	18

	4.4.1	Use Case Diagram	18-19
	4.4.2	Class Diagrams	19-20
	4.4.3	Sequence Diagram	20-21
	4.4.4	Activity Diagram	21-22
	4.4.5	Component Diagram	22-23
	4.4.6	Deployment Diagram	24
4.5	Database Design		25
	4.5.1	ER Diagram	25-26
5.	**IMPLEMENTATION**		27-29
5.1	Software Used		30-31
5.2	Algorithm		32-34
5.3	Source Code		34-43
6.	**TESTING**		
6.1	Introduction		44
6.2	Levels of Testing		45
6.3	Test Cases		46-51
7.	**RESULTS AND OUTPUT**		
7.1	Output Screens		52-56
7.2	Results		56
8.	**CONCLUSION**		57
9.	**FUTURE SCOPE**		58
10.	**REFERENCE**		59-60

LIST OF FIGURES

S.no	NAME OF FIGURE		PAGE No
1	Fig 1.1	Deepfake Manipulation	2
2	Fig 1.2	Proposed Solution	6
3	Fig 4.1	System Architecture	14
4	Fig 4.2	System Architecture of Deepfake	15
5	Fig 4.3	System Methodology	17
6	Fig 4.4	Use Case Diagram	19
7	Fig 4.5	Class Diagram	20
8	Fig 4.6	Sequence Diagram	21
9	Fig 4.7	Activity Diagram	22
10	Fig 4.8	Component Diagram	23
11	Fig 4.9	Deployment Diagram	24
12	Fig 4.10	Entity Relationship Diagram	26
13	Fig 5.1	Video DataSet	31
14	Fig 5.2	File Structure	43
15	Fig 6.1	Video Upload	46
16	Fig 6.2	Prediction Result	47
17	Fig 6.3	SHAP Visualization	48
18	Fig 6.4	Frame Importance	49
19	Fig 6.5	Grad-CAM Visualization	50
20	Fig 6.6	File Type Restriction	51
15	Fig 7.1	Home Page	52
16	Fig 7.2	Video Upload and Preview	53
17	Fig 7.3	Prediction Result	53
18	Fig 7.4	SHAP Values Visualization	54

19	Fig 7.5	Frame Importance Graph	55
20	Fig 7.6	Grad-CAM Visualization	56

LIST OF TABLES

S.no	NAME OF TABLES		PAGE No
1	Table 6.1	Video Upload	46
2	Table 6.2	Prediction Result	47
3	Table 6.3	SHAP Visualization	48
4	Table 6.4	Frame Importance	48
5	Table 6.5	Grad-CAM Visualization	49
6	Table 6.6	File Type Restriction	50

CHAPTER 1
INTRODUCTION

1. INTRODUCTION

Deepfake technology has rapidly evolved with the advancement of artificial intelligence and deep learning, enabling the creation of highly realistic manipulated videos and images. These fake media contents can easily deceive people and are increasingly being used to spread misinformation, influence public opinion, and perform digital fraud. Therefore, deepfake detection has become a critical task in ensuring the authenticity and reliability of digital media.

Deepfake Detection involves identifying whether a video or image is real or artificially manipulated. This process is mainly carried out using Computer Vision techniques, where systems analyze visual data such as images and video frames. Convolutional Neural Networks (CNNs) play a major role in this process by extracting important features from images. In this project, a pretrained CNN model, InceptionV3, is used for feature extraction, as it is effective in capturing high-level spatial patterns from facial regions.

Although deep learning models provide high accuracy, most existing systems function as black boxes, offering predictions without explaining the reasoning behind them. To overcome this limitation, Explainable Artificial Intelligence (XAI) is incorporated into the system to improve transparency and user trust. XAI techniques help users understand how the model makes decisions.

In this system, Grad-CAM (Gradient-weighted Class Activation Mapping) is used to generate heat maps that highlight important regions of the face influencing the prediction. Similarly, SHAP (SHapley Additive exPlanations) is used to determine the contribution of each frame or feature to the final decision. These techniques provide clear visual and analytical explanations, making the system more interpretable.

The proposed Intelligent Human Face Deepfake Detection System integrates deep learning and explainable AI techniques to accurately classify videos as real or fake while providing meaningful insights into the prediction process. This approach improves both the accuracy and transparency of deepfake detection, making it a reliable and user-friendly solution for combating digital misinformation.

Fig 1.1 Deepfake manipulation

1.1 PROBLEM DEFINITION

The rapid growth of deepfake technology has made it increasingly difficult to distinguish between real and manipulated images and videos. Deepfakes are capable of creating highly realistic fake media that can mislead people, spread misinformation, damage reputations, and pose serious threats to security and privacy.

Existing deepfake detection systems mainly focus on achieving high accuracy using deep learning models, but most of them operate as black-box systems. They provide only the final prediction without explaining how the decision was made. This lack of transparency reduces user trust and makes it difficult to verify or justify the results, especially in critical applications.

Manual verification of digital media is time-consuming, error-prone, and not feasible for large-scale platforms. Moreover, many existing systems fail to clearly identify which regions of the image or video are manipulated, limiting their effectiveness in real-world scenarios.

Therefore, there is a need to develop an intelligent deepfake detection system that not only accurately detects fake media but also provides clear and interpretable explanations for its predictions, making the system more reliable, transparent, and user-friendly.

1.2 PURPOSE OF THE SYSTEM

The main purpose of this system is to develop an efficient and reliable solution for detecting deepfake videos and distinguishing them from real content. With the increasing use of artificial intelligence in media manipulation, there is a growing need for automated systems that can accurately identify fake videos without human intervention. This system aims to analyze video content by extracting meaningful features and identifying inconsistencies that indicate manipulation.

Another important purpose of the system is to improve transparency and trust in deepfake detection by integrating explainable artificial intelligence techniques. Unlike traditional models that act as black boxes, this system provides visual and analytical

explanations, such as highlighting manipulated regions and identifying important frames that influence the prediction. This helps users understand how and why a particular video is classified as real or fake.

1.3 SCOPE OF THE SYSTEM

The scope of this project is to develop an intelligent system capable of detecting deepfake images and videos using advanced deep learning techniques. The system focuses on analyzing human facial features by extracting meaningful spatial and temporal patterns from video frames to accurately classify content as real or fake.

The project includes the use of a pre-trained InceptionV3 model for feature extraction and a sequence model to analyze temporal inconsistencies across frames. It also integrates Explainable Artificial Intelligence techniques such as Grad-CAM and gradient-based methods to provide visual explanations by highlighting manipulated facial regions and important frames.

The system is trained and evaluated using benchmark datasets such as FaceForensics++, DFDC, and Celeb-DF to ensure reliable performance. A user-friendly web interface is developed to allow users to upload videos and view prediction results along with confidence scores and explanation heatmaps.

This project is designed for real-world applications such as media verification, cybersecurity, digital forensics, and social media content analysis. However, the scope is limited to human face-based deepfake detection and may not cover other types of synthetic media or non-facial manipulations.

1.4 MOTIVATION BEHIND THE PROBLEM SELECTION

Deepfake technology has emerged as a powerful application of artificial intelligence, enabling the creation of highly realistic manipulated videos and images. While it has useful applications in entertainment and media, it also poses serious threats such as misinformation, identity theft, cybercrime, and manipulation of public opinion. The increasing misuse of deepfakes on social media platforms and digital communication has made it difficult for people to trust the authenticity of online content.

Most existing deepfake detection systems focus only on accuracy and operate as black-box models, providing results without explaining how the decision was made. This lack of transparency reduces user confidence and makes it difficult to rely on these systems in sensitive domains such as digital forensics and security.

The motivation behind selecting this problem is to develop a system that not only detects deepfakes accurately but also explains the reasoning behind its predictions. By integrating

Explainable Artificial Intelligence techniques, the system aims to improve transparency, build user trust, and make deepfake detection more reliable and understandable. This project also addresses the growing need for practical and user-friendly tools that can help individuals and organizations combat the spread of fake media in today's digital world.

1.5 EXISTING SYSTEM

In the current scenario, detecting deepfake videos is primarily done using basic visual inspection or traditional digital forensics techniques. These methods often rely on manual scrutiny, pixel-level analysis, or simple machine learning models that examine inconsistencies in individual frames. Such approaches are time-consuming, error-prone, and limited in accuracy, especially when manipulations are subtle or distributed across multiple frames.

Many existing systems lack the capability to analyze temporal dependencies, making them ineffective at capturing subtle changes in facial expressions, head movements, or lip-sync errors that occur over a sequence of frames. Moreover, most conventional systems do not provide explainable outputs, leaving users unaware of why a video was classified as fake or real, which reduces trust and transparency.

- **Drawbacks of Existing System**

a. Existing systems take more time and often require manual checking, which makes the process slow and less reliable.

b. They mainly analyze individual frames and do not consider the relationship between multiple frames, leading to poor detection of complex deepfakes.

c. These systems cannot effectively detect small changes in facial expressions, head movements, or lip synchronization.

d. Their performance decreases when videos are of low quality or highly compressed.

e. Many existing models do not work well for new types of deepfake techniques.

f. Most systems do not explain their results, so users cannot understand why a video is classified as real or fake.

g. They do not provide visual outputs like highlighted regions or important frames, which reduces clarity and user trust.

1.6 PROPOSED SYSTEM

The proposed system is an **Intelligent Human Face Deepfake Detection System using Explainable Artificial Intelligence (XAI)** that automates the process of detecting fake videos while providing transparent reasoning behind each prediction. Unlike traditional methods, this system leverages deep learning and temporal analysis to identify subtle manipulations across multiple video frames, improving both accuracy and reliability.

The system processes uploaded videos by extracting a fixed number of frames and performing face detection using OpenCV Haar Cascade classifiers, followed by resizing and normalization. High-level spatial features are extracted from each frame using a pretrained InceptionV3 convolutional neural network, producing a 2048-dimensional feature vector. These features are then fed into a sequence model that captures temporal dependencies to accurately classify the video as real or fake.

To enhance interpretability, the system incorporates explainable AI techniques such as Grad-CAM for visualizing manipulated facial regions and SHAP values for analyzing frame-level importance. This approach ensures high detection accuracy, robustness against subtle deepfakes, and transparency in decision-making, making it a practical and trustworthy tool for combating digital misinformation.

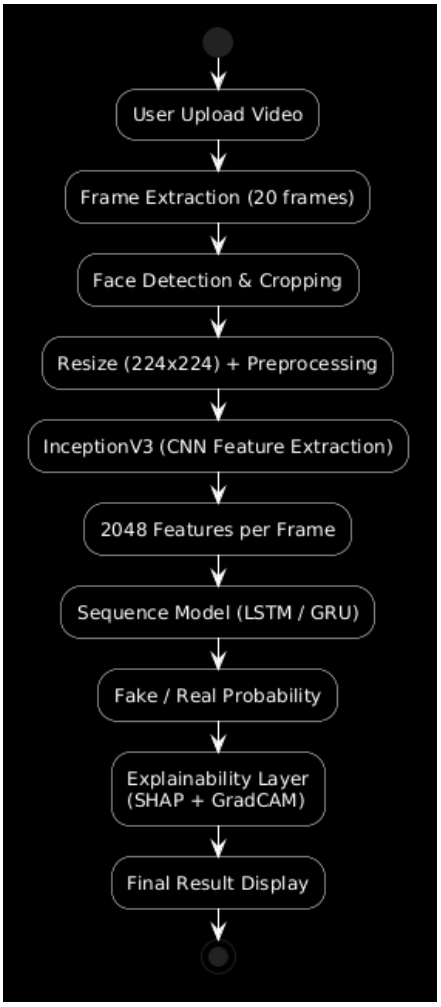

Fig 1.2 Proposed Solution

CHAPTER 2
REQUIREMENT ANALYSIS

REQUIREMENT ANALYSIS

The development of the intelligent deepfake human face detection system begins with a careful analysis of the requirements needed to ensure its effectiveness, accuracy, and usability. The system must allow users to upload videos and automatically process them by extracting frames and detecting faces. Each frame should undergo feature extraction using a pretrained InceptionV3 model, and the temporal dependencies across frames should be analyzed by a sequence model to classify the video as real or fake. In addition, the system should provide explainable outputs, including Grad-CAM visualizations and SHAP-based frame importance, to help users understand the basis of the predictions.

Beyond functional capabilities, the system must meet several non-functional requirements. It should operate efficiently, processing videos in a reasonable amount of time while maintaining high accuracy and reliability, even when manipulations are subtle. The interface must be user-friendly to ensure smooth interaction, and the system should be scalable to handle videos of varying lengths and resolutions. Furthermore, security and privacy of uploaded videos must be maintained throughout the processing pipeline.

By addressing both functional and non-functional requirements, the system can deliver a robust, interpretable, and practical solution for detecting deepfake videos and enhancing user trust in media verification.

2.1 FUNCTIONAL REQUIREMENTS

- The system must allow users to upload videos through an interactive interface.
- It should perform frame extraction from the uploaded videos.
- The system must carry out face detection and preprocessing (resizing and normalization) on each frame.
- High-level features must be extracted from each frame using a pretrained InceptionV3 model.
- The system should analyze the temporal sequence of features to classify videos as real or fake.
- It must provide explainable outputs, including Grad-CAM visualizations and SHAP-based frame importance.
- Users should be able to view prediction results and interpret visual explanations clearly.

2.2 NON-FUNCTIONAL REQUIREMENTS

- The system shall provide high accuracy in detecting deepfake videos, even for subtle manipulations.
- The system shall process videos efficiently within a reasonable time to support near real-time performance.
- The system shall have a user-friendly interface that is easy to use for both technical and non-technical users.
- The system shall be reliable and produce consistent results for repeated inputs.
- The system shall be scalable to handle videos of different lengths and resolutions.
- The system shall ensure the security and privacy of uploaded videos during processing.
- The system shall provide clear and interpretable explanations to improve transparency and user trust.
- The system shall be maintainable and allow future updates or improvements to the model.

2.3 SOFTWARE REQUIREMENTS

- Operating System: Windows / Linux / macOS
- Programming Language: Python 3.x
- Development Environment: VS Code
- Libraries and Frameworks: TensorFlow, Keras, OpenCV, NumPy, Pandas, Streamlit
- Dataset Sources: Video datasets (real and deepfake videos, MP4 format)
- Diagram Tools: PlantUML
- Presentation Tool: Microsoft PowerPoint

2.4 HARDWARE REQUIREMENTS

- Hardware Processor: Intel Core i5 / AMD Ryzen 5 or higher
- RAM: 8 GB minimum (16 GB recommended)
- Storage: 256 GB SSD or higher
- Graphics: Integrated GPU (Dedicated GPU optional)
- Display: Minimum 1366 × 768 resolution
- Network: Stable Internet connection for dataset access
- Input Devices: Keyboard and Mouse

CHAPTER 3
SYSTEM ANALYSIS

3.1 INTRODUCTION

The system analysis focuses on understanding the limitations of existing deepfake detection approaches and identifying the requirements for an improved solution. Current methods often struggle with low accuracy, inability to capture temporal dependencies across video frames, and lack of explainable outputs, which reduces user trust.

To overcome these limitations, this project defines key functional requirements, including automated video upload, frame extraction, face detection, feature extraction using InceptionV3, sequence modeling for temporal analysis, and explainable AI techniques such as Grad-CAM and SHAP. Additionally, non-functional requirements such as efficiency, reliability, scalability, and user-friendliness are considered.

The analysis also assesses technical feasibility, hardware and software needs, and guides the design approach for developing a robust, interpretable, and user-friendly deepfake detection system. This structured analysis ensures that the proposed system is practical, effective, and capable of addressing the limitations of current approaches.

3.2 LITERATURE SURVEY

[1] A. Rössler et al., *"FaceForensics++: Learning to Detect Manipulated Facial Images"*, IEEE ICCV, 2019.

FaceForensics++ is one of the most widely used datasets for deepfake detection. It provides a large collection of real and manipulated videos generated using different face manipulation techniques. The dataset helps in training deep learning models to identify visual artifacts and inconsistencies in facial regions. This work highlights the importance of high-quality datasets in improving the performance of deepfake detection systems.

[2] B. Dolhansky et al., *"The DeepFake Detection Challenge Dataset"*, CVPR Workshops, 2020.

The DeepFake Detection Challenge (DFDC) dataset contains a diverse set of videos with various lighting conditions, facial expressions, and compression levels. This dataset improves the robustness of detection systems by exposing models to real-world variations. It plays a significant role in enhancing the generalization capability of deepfake detection models.

[3] Y. Li et al., *"Celeb-DF: A Large-Scale Dataset for DeepFake Detection"*, CVPR, 2020.

Celeb-DF is designed to overcome the limitations of earlier datasets by providing more realistic deepfake videos with fewer visual artifacts. It challenges detection models to identify subtle manipulations, making it suitable for evaluating real-world performance of deepfake detection systems.

[4] C. Szegedy et al., *"Rethinking the Inception Architecture for Computer Vision"*, CVPR, 2016.

InceptionV3 is a powerful convolutional neural network architecture used for image classification and feature extraction. It improves computational efficiency while maintaining high accuracy. In deepfake detection, this model is used to extract high-level spatial features from facial images, which helps in identifying inconsistencies.

[5] R. R. Selvaraju et al., *"Grad-CAM: Visual Explanations from Deep Networks"*, ICCV, 2017.

Grad-CAM is an explainable AI technique that generates heatmaps highlighting important regions in an image that influence model predictions. In deepfake detection, it is used to visualize manipulated facial regions, improving transparency and interpretability of the model.

[6] S. Lundberg and S.-I. Lee, *"A Unified Approach to Interpreting Model Predictions"*, NeurIPS, 2017.

SHAP is a method used to explain machine learning model predictions by assigning importance values to input features. It helps in understanding how each feature contributes to the final decision, making the model more interpretable and trustworthy.

[7] I. Goodfellow et al., *"Generative Adversarial Networks"*, NeurIPS, 2014.

Generative Adversarial Networks (GANs) are the foundation of deepfake technology. They consist of generator and discriminator networks that work together to create realistic fake images and videos. Understanding GANs is essential for developing effective deepfake detection systems.

[8] Y. Mirsky and W. Lee, *"The Creation and Detection of Deepfakes: A Survey"*, ACM Computing Surveys, 2021.

This survey provides a comprehensive overview of deepfake generation and detection techniques. It discusses various challenges such as generalization, robustness, and the need for explainability in detection systems.

[9] R. Tolosana et al., *"DeepFakes and Beyond: A Survey of Face Manipulation Detection"*, Information Fusion, 2020.

This work reviews different deepfake detection approaches, including biometric-based methods and deep learning techniques. It highlights the importance of combining spatial and temporal analysis for improved detection accuracy.

[10] D. Afchar et al., *"MesoNet: A Compact Facial Video Forgery Detection Network"*, WIFS, 2018.

MesoNet is a lightweight deep learning model designed for detecting facial forgeries. It focuses on detecting mesoscopic-level features and provides efficient performance with reduced computational complexity.

3.3 FEASIBILITY STUDY

Feasibility study is carried out to determine whether the proposed system is practical and can be successfully implemented within the available resources and constraints. It evaluates different aspects such as technical, economic, and social feasibility.

3.3.1 TECHNICAL FEASIBILITY

The proposed system is technically feasible as it uses widely available technologies such as Python, TensorFlow, Keras, OpenCV, and Streamlit. These tools support deep learning, video processing, and web application development. The system can be implemented using standard computing resources without requiring specialized hardware, although a GPU can improve performance. The availability of datasets such as FaceForensics++ and DFDC further supports model training and evaluation.

3.3.2 ECONOMICAL FEASIBILITY

The system is economically feasible as it is developed using open-source tools and libraries, which reduces development costs. No expensive software licenses are required. The hardware requirements are moderate and affordable, making the system cost-effective for development and deployment.

3.3.3 SOCIAL FEASIBILITY

The system is socially beneficial as it helps in detecting fake media and preventing the spread of misinformation. It improves digital security, protects individuals from identity misuse, and enhances trust in online content. The user-friendly interface ensures that both technical and non-technical users can easily use the system.

CHAPTER 4
SYSTEM DESIGN

4.1 INTRODUCTION

The system design of the Intelligent Deepfake Detection System is structured to efficiently process videos, detect manipulations, and provide explainable outputs. The architecture is divided into five main modules: Preprocessing, Feature Extraction, Classification, Explainability, and User Interface. In the preprocessing module, uploaded videos are broken down into a fixed number of frames, and faces are detected and normalized using OpenCV.

The feature extraction module employs a pretrained InceptionV3 model to generate high-level spatial features for each frame, producing a 2048-dimensional vector. These features are then fed into a temporal sequence model, such as LSTM, to capture temporal dependencies across frames and classify videos as real or fake.

The explainability module integrates Grad-CAM and SHAP techniques to highlight important frames and regions influencing the model's predictions, enhancing transparency and user trust. Finally, the user interface module, built with Streamlit, allows users to upload videos, view classification results, and interpret visual explanations easily.

The design ensures modularity, scalability, and maintainability, allowing individual components to be updated or replaced without affecting the overall system. This structured approach provides an effective, accurate, and interpretable solution for deepfake detection.

4.2 SYSTEM ARCHITECTURE

The system architecture of the Intelligent Human Face Deepfake Detection System represents the overall workflow of the system, illustrating how input data is processed, analyzed, and converted into meaningful outputs with explainability.

The process begins with the input stage, where the user uploads an image or video through the web interface. The input is then passed to the preprocessing module, where the video is divided into frames, and face detection is performed using OpenCV to extract relevant facial regions.

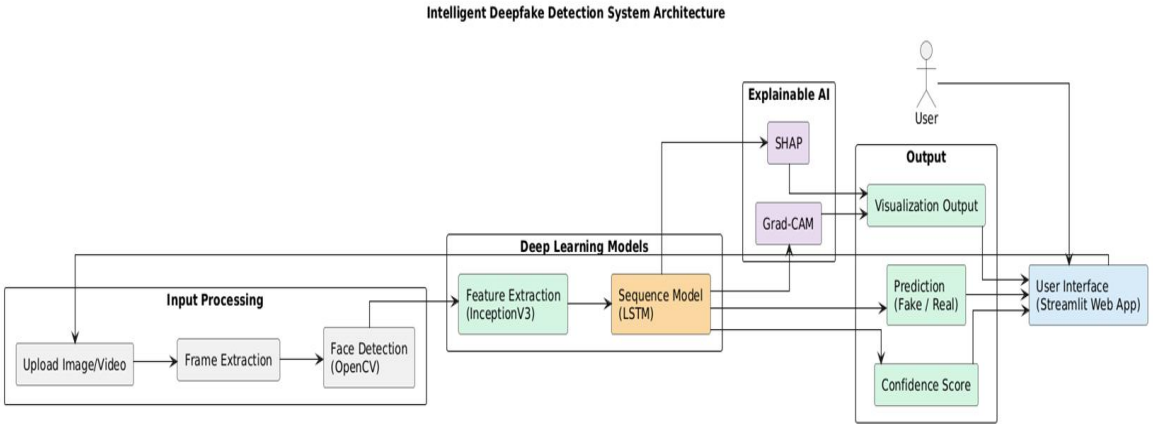

Fig 4.1 System Architecture

In the deep learning module, feature extraction is carried out using a pre-trained InceptionV3 model, which generates high-level spatial features from each frame. These features are then passed to a sequence model, such as LSTM, which analyzes temporal dependencies across frames to classify the input as real or fake.

The system also includes an Explainable AI module, where techniques such as Grad-CAM and SHAP are applied. Grad-CAM highlights manipulated facial regions, while SHAP identifies the importance of different frames in the prediction process.

Finally, the output module displays the classification result (fake or real), along with a confidence score and visual explanations. The results are presented through a user-friendly interface built using Streamlit, allowing users to easily understand and interpret the predictions.

This architecture ensures efficient processing, accurate detection, and improved transparency, making the system reliable and suitable for real-world applications.

The system architecture is organized into multiple layers, each responsible for a specific function in the deepfake detection process.

1. Input Layer

- **Video Upload:** Users provide a video file through the Streamlit interface.
- **Video Preprocessing:** The uploaded video is divided into frames for further analysis.

2. Face Detection & Preprocessing Layer

- **Face Detection:** OpenCV Haar Cascade is used to detect faces in each frame.
- **Face Cropping & Resizing:** Detected faces are cropped and resized to a fixed dimension suitable for the neural network.
- **Normalization:** Pixel values are normalized to prepare the data for feature extraction.

3. Feature Extraction Layer

- **Pre-trained CNN (InceptionV3):** Extracts high-level spatial features from each frame.
- **Output:** Generates a 2048-dimensional feature vector for each frame representing facial features.

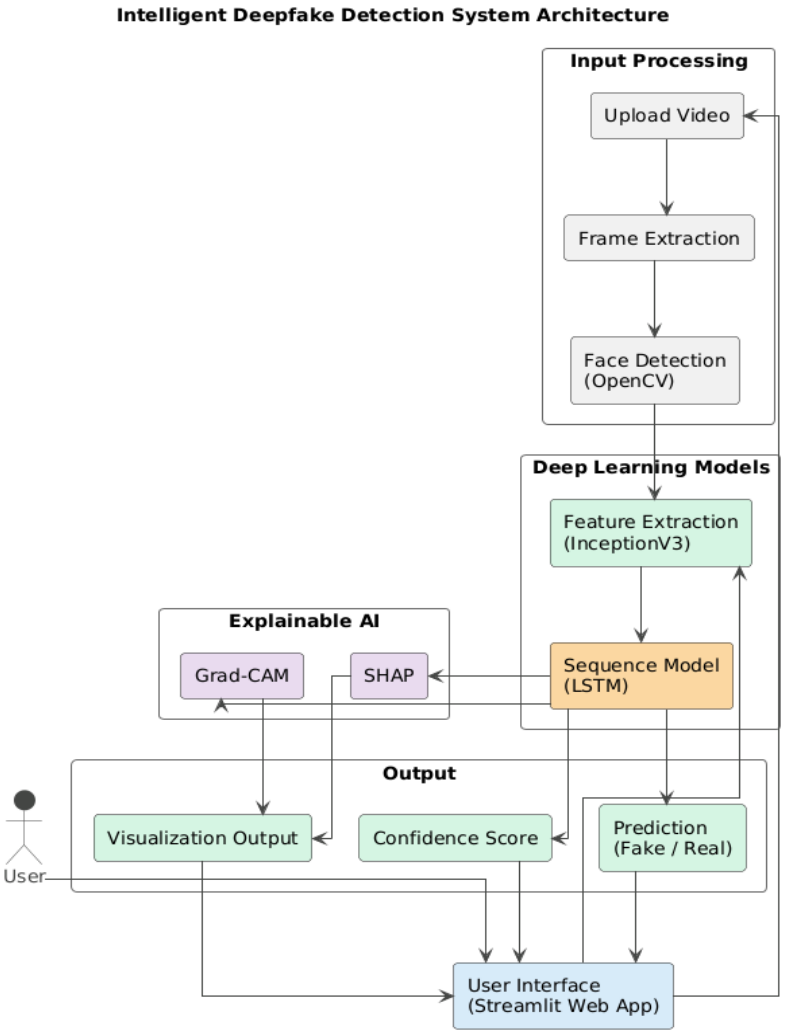

Fig 4.2 System Architecture of Deepfake Detection System

4. Temporal Analysis Layer

- **Sequence Model (LSTM):** Analyzes the sequence of frame features to capture temporal inconsistencies.
- **Prediction:** Produces the probability of the video being real or fake.

5. Explainable AI Layer

- **Grad-CAM Visualization:** Highlights important facial regions influencing the prediction.
- **SHAP Values:** Provides frame-level importance to show which frames contributed most to the decision.

6. User Interface Layer

- **Streamlit Dashboard:** Displays the final results to the user, including:
  - Prediction result (Real or Fake)
  - Confidence score
  - Grad-CAM heatmaps
  - SHAP-based interpretability graphs
  - Interactive visualization for better understanding

4.3 SYSTEM METHODOLOGY

The proposed Intelligent Deepfake Detection System follows a structured approach to analyze video content and identify whether it is real or manipulated. The process begins with the user uploading a video through a Streamlit-based interface. The uploaded video is then converted into a fixed number of frames at regular intervals to capture relevant visual information.

Each extracted frame undergoes face detection using OpenCV Haar Cascade, where the facial region is identified, cropped, and resized to a standard dimension. The frames are normalized to ensure consistency in input data. These preprocessed frames are then passed to a pre-trained InceptionV3 model, which extracts high-level spatial features and represents each frame as a 2048-dimensional feature vector.

The extracted feature vectors are fed into a sequence model that analyzes temporal dependencies across frames to identify inconsistencies. Based on this analysis, the system classifies the video as real or fake and generates a confidence score. To enhance transparency, Explainable AI techniques such as Grad-CAM and SHAP are applied to highlight important regions and frames influencing the prediction. Finally, the results are displayed through a user-friendly interface, allowing users to interpret the output easily.

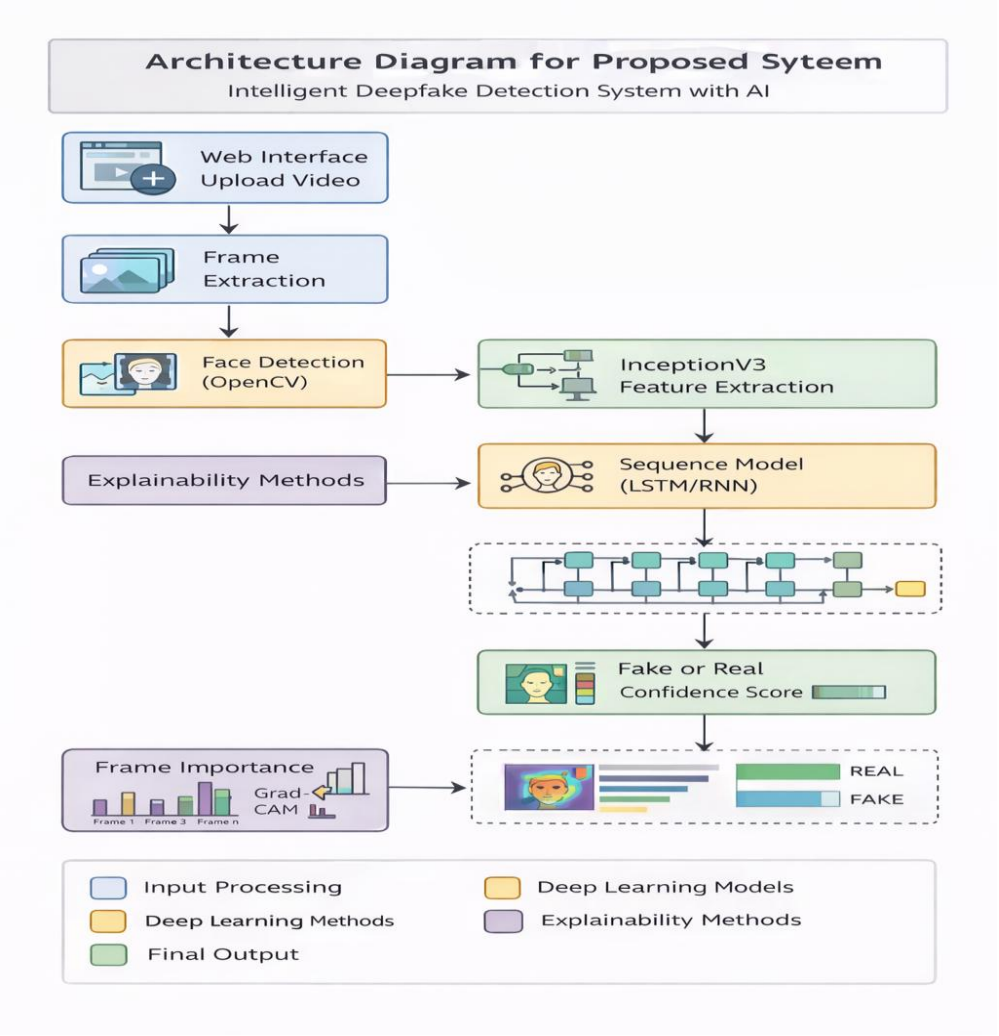

Fig 4.3 System Methodology

The system **methodology** describes the step-by-step process followed to detect deepfake videos and provide explainable results. The overall workflow of the system is as follows:

1. **Video Input:** User uploads a video through the web interface.
2. **Frame Extraction:** Video is divided into frames at regular intervals.

3. **Face Detection:** Faces are detected using OpenCV and extracted from frames.
4. **Preprocessing:** Faces are resized and normalized for model input.
5. **Feature Extraction:** InceptionV3 extracts spatial features (2048-dimensional vectors).
6. **Temporal Analysis:** Sequence model analyzes frame features over time.
7. **Prediction:** System classifies the video as real or fake with a confidence score.
8. **Explainability:** Grad-CAM and SHAP highlight important regions and frames.
9. **Output Display:** Results and visual explanations are shown in the interface.

4.4 UML DIAGRAMS

Unified Modeling Language (UML) diagrams are used to represent the structure, behavior, and interactions of the system in a visual manner. These diagrams help in understanding the system design clearly and provide a blueprint for implementation. In this project, various UML diagrams are used to describe different aspects of the Intelligent Human Face Deepfake Detection System.

4.4.1 USE CASE DIAGRAM

The Use Case Diagram represents the interaction between different users and the Intelligent Deepfake Detection System. It illustrates how various actors utilize the system to perform specific tasks.

The primary actor is the User, who interacts with the system by uploading a video for analysis. Once the video is uploaded, the system processes the video and generates a prediction indicating whether the content is real or fake. The user can also view the prediction results along with explainable outputs such as Grad-CAM visualizations and SHAP-based explanations, which help in understanding how the decision was made.

The Admin actor is responsible for managing the system's backend operations. This includes maintaining the dataset and updating the deep learning model to improve system performance and accuracy.

The Researcher actor focuses on analyzing the results generated by the system. This helps in evaluating model performance, studying patterns in deepfake detection, and improving future models.

Overall, the Use Case Diagram provides a clear understanding of how different users interact with the system and highlights the key functionalities offered by the proposed solution.

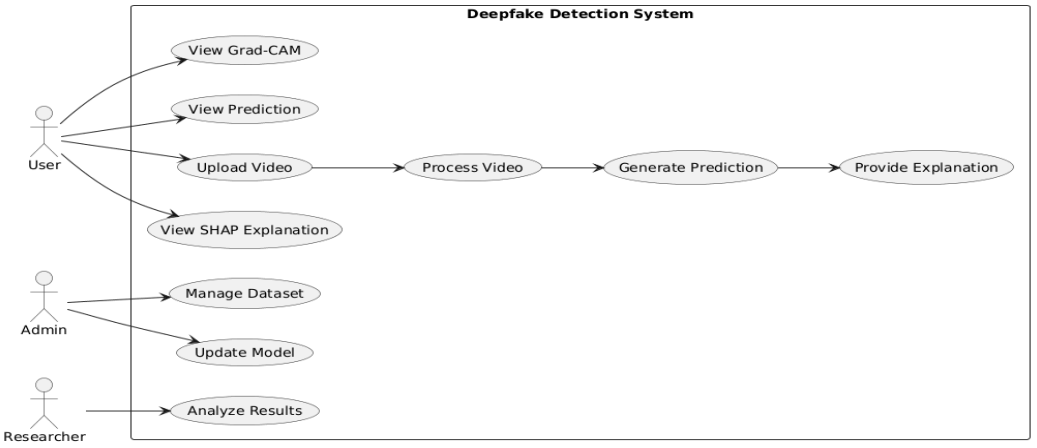

Fig 4.4 Use Case Diagram

4.4.2 CLASS DIAGRAMS

The class diagram represents the structure of the system by illustrating various classes, their attributes, methods, and relationships. The main classes in the Intelligent Deepfake Detection System include User, StreamlitInterface, Video, Frame, Face, FeatureExtractor, SequenceModel, and ExplainableAI.

The User class interacts with the system through the StreamlitInterface, which handles operations such as media upload, prediction display, and visualization of explanations. The Video class manages video-related data and performs frame extraction, while the Frame class processes individual frames and detects faces. The Face class stores cropped facial regions for further analysis.

The FeatureExtractor class utilizes a pre-trained InceptionV3 model to extract meaningful spatial features from face images. These features are then passed to the SequenceModel class, which captures temporal dependencies across frames and generates the final prediction indicating whether the video is real or fake.

To enhance transparency, the ExplainableAI class produces interpretability outputs such as Grad-CAM heatmaps and SHAP values, helping users understand the reasoning behind the model's decision.

Overall, the class diagram illustrates how different components of the system interact to process input data and generate accurate and explainable results.

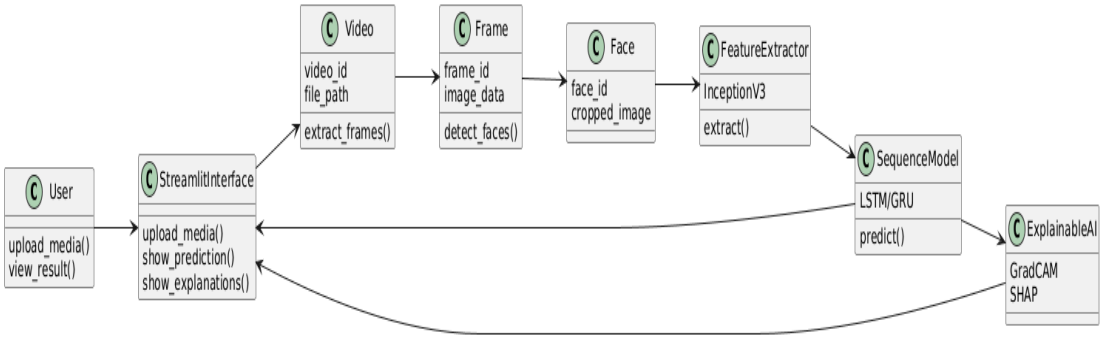

Fig 4.5 Class Diagram

4.4.3 SEQUENCE DIAGRAM

The sequence diagram illustrates the flow of interactions between different components of the Intelligent Deepfake Human Face Detection System over time. It shows how data moves from the user to various processing modules and finally produces the output.

The process begins when the user uploads a video through the Streamlit user interface. The video is then passed to the video processing module, where frames are extracted. These frames are sent to the face detection module, which identifies and extracts facial regions using OpenCV.

The detected faces are forwarded to the feature extraction module, where a pre-trained InceptionV3 model extracts high-level features from each frame. These features are then passed to the sequence model, such as LSTM, which analyzes temporal relationships across frames to determine whether the video is real or fake.

After classification, the prediction is sent back to the user interface. Simultaneously, the explainable AI module generates visual explanations using techniques such as Grad-CAM and SHAP, highlighting important regions and frames influencing the decision.

Finally, the system displays the prediction result along with the explanation outputs to the user, completing the interaction process.

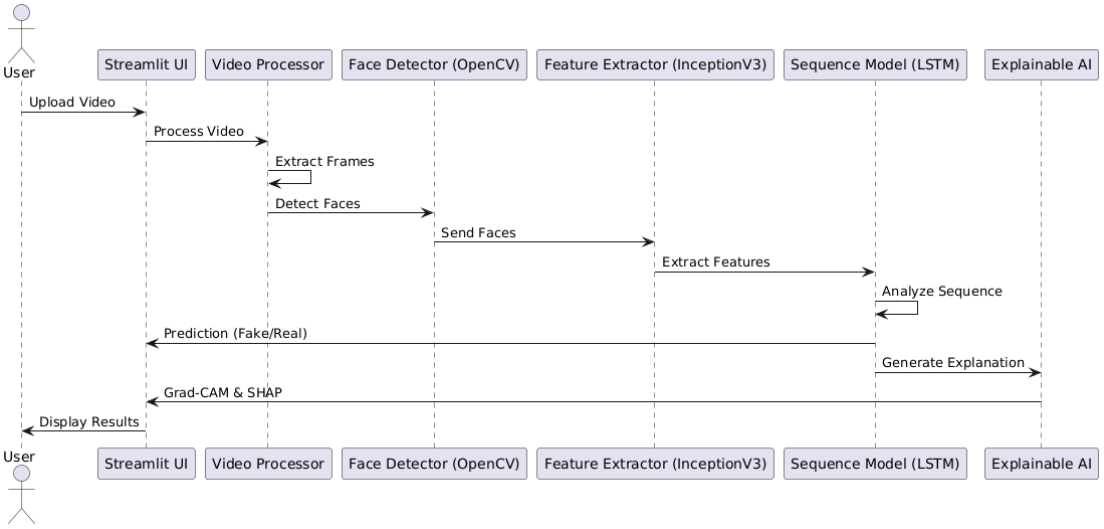

Fig 4.6 Sequence Diagram

4.4.4 ACTIVITY DIAGRAMS

Activity Flow Points: Intelligent Human Face Deepfake Detection System

1. **Start:** The user uploads a video to the system through the interface.
2. **Frame Extraction:** The uploaded video is processed to extract individual frames.
3. **Face Detection:** Each frame is analyzed using OpenCV Haar Cascade to detect facial regions.
4. **Face Preprocessing:** Detected faces are cropped, resized to a standard input size, and normalized for model processing.
5. **Feature Extraction:** Each preprocessed face is passed through a pre-trained InceptionV3 model to extract 2048-dimensional feature vectors.
6. **Sequence Modeling:** The extracted features are fed into a sequence model (LSTM/GRU) to capture temporal dependencies across frames.
7. **Prediction:** The sequence model generates a probability score to classify the video as real or fake.
8. **Explainable AI Outputs:**

    - Generate Grad-CAM visualizations to highlight facial regions influencing the prediction.

- Compute SHAP values to indicate contributions of each frame to the final decision.

9. **Result Display:** The system presents the final prediction along with visual explanations through the Streamlit interface.
10. **End:** The user reviews and interprets the results.

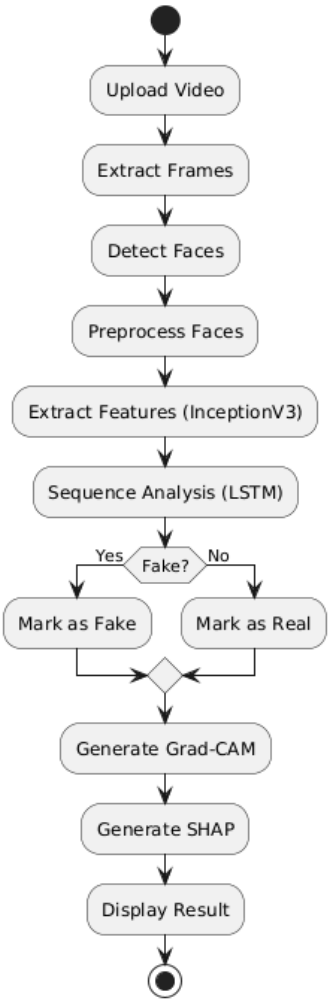

Fig 4.7 Activity Diagram

4.4.5 COMPONENT DIAGRAM

The component diagram represents the high-level organization of the Intelligent Deepfake Detection System by illustrating the major system modules and their interactions. It shows how different components work together to process input data and generate the final output.

The system begins with the User Interface component, developed using Streamlit, which allows users to upload videos and view results. The uploaded video is passed to the Video Processing Module, where frames are extracted for further analysis.

The frames are then sent to the Face Detection component, implemented using OpenCV, which identifies and extracts facial regions. These detected faces are forwarded to the Feature Extraction module, where a pre-trained InceptionV3 model extracts high-level features from each frame.

The extracted features are processed by the Sequence Model (LSTM), which analyzes temporal dependencies across frames to classify the video as real or fake. The prediction results are then sent to the Prediction Output component.

To improve interpretability, the Explainable AI Module is integrated into the system. This module generates visual explanations using Grad-CAM and SHAP, highlighting important regions and frames that influence the model's decision.

Finally, all results, including predictions and explanations, are displayed to the user through the Streamlit interface. The component diagram clearly illustrates the modular structure of the system, ensuring scalability, maintainability, and efficient data flow.

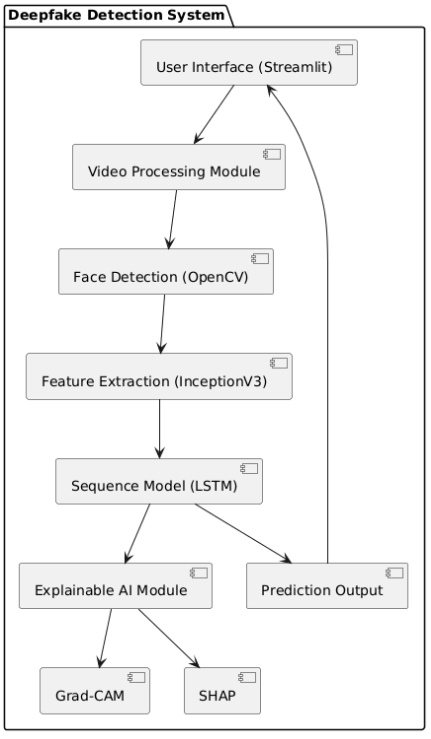

Fig 4.8 Component Diagram

4.4.6 Deployment Diagram

The deployment diagram represents the physical architecture of the Intelligent Deepfake Detection System, showing how software components are deployed on hardware devices and how they interact during execution.

The system consists of two main entities: the user device and the server. The user interacts with the system through a web browser on their device. The browser sends requests to the server, where the main application is hosted.

The server runs the Streamlit application, which acts as the central interface for handling user inputs and displaying results. Within the server environment, several components are deployed to perform different tasks. The OpenCV processing module is responsible for video preprocessing tasks such as frame extraction and face detection.

The deep learning model component handles feature extraction and classification using models such as InceptionV3 and LSTM. The explainable AI module is responsible for generating interpretability outputs using techniques such as Grad-CAM and SHAP.

All these components work together within the server to process the uploaded video and generate the final prediction. The results, including classification and explanations, are then sent back to the user's browser for display.

This deployment structure ensures efficient processing, centralized computation, and easy accessibility for users through a web-based interface.

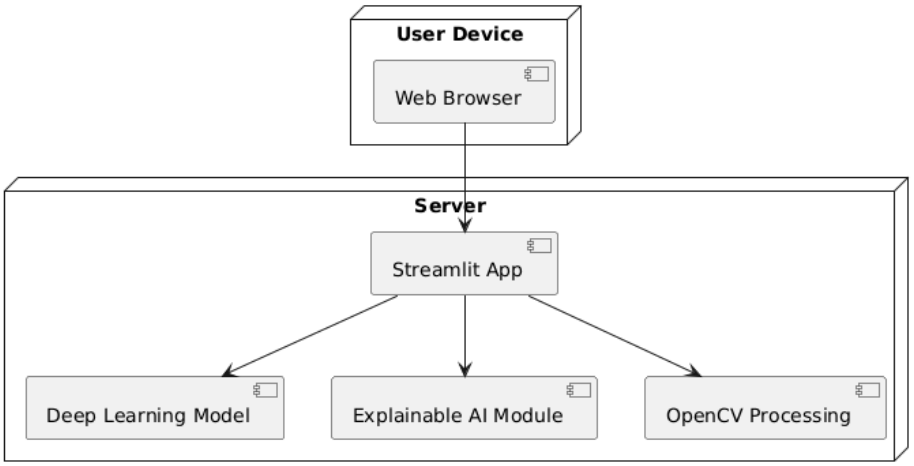

Fig 4.9 Deployment Diagram

4.5 Database Design

Database design defines the structure for storing and managing data in the system. It organizes data into entities, attributes, and relationships to ensure efficient storage, retrieval, and consistency. The Entity Relationship (ER) diagram is used to represent the database structure of the proposed system.

4.5.1 Entity Relationship Diagrams

The database for the Intelligent Deepfake Detection System is designed to efficiently store and manage all information related to users, uploaded videos, extracted frames and faces, feature vectors, predictions, and explainability outputs. The design ensures data retrieval, maintains data integrity, and avoids redundancy through a structured relational model.

The Entity Relationship Diagram (ERD) illustrates the structure and relationships of the system's database entities:

1. **User** –A single user can upload multiple videos.
2. **Video** – Stores details of uploaded videos, including VideoID (primary key), file path, upload time, and type. Each video consists of multiple frames.
3. **Frame** – Contains extracted frames with FrameID (primary key) and VideoID (foreign key). Each frame represents a portion of the video and may contain multiple faces.
4. **Face** – Stores detected faces with FaceID (primary key) and FrameID (foreign key). Each face is extracted from a frame for further analysis.
5. **FeatureVector** – Stores extracted features with FeatureID (primary key) and FaceID (foreign key). It represents the high-level features obtained from the deep learning model.
6. **Prediction** – Stores classification results with PredictionID (primary key), VideoID (foreign key), prediction result (real/fake), and confidence score.
7. **Explainability** – Contains explainable AI outputs such as Grad-CAM and SHAP, with ExplainID (primary key) and PredictionID (foreign key), linking explanations to prediction.

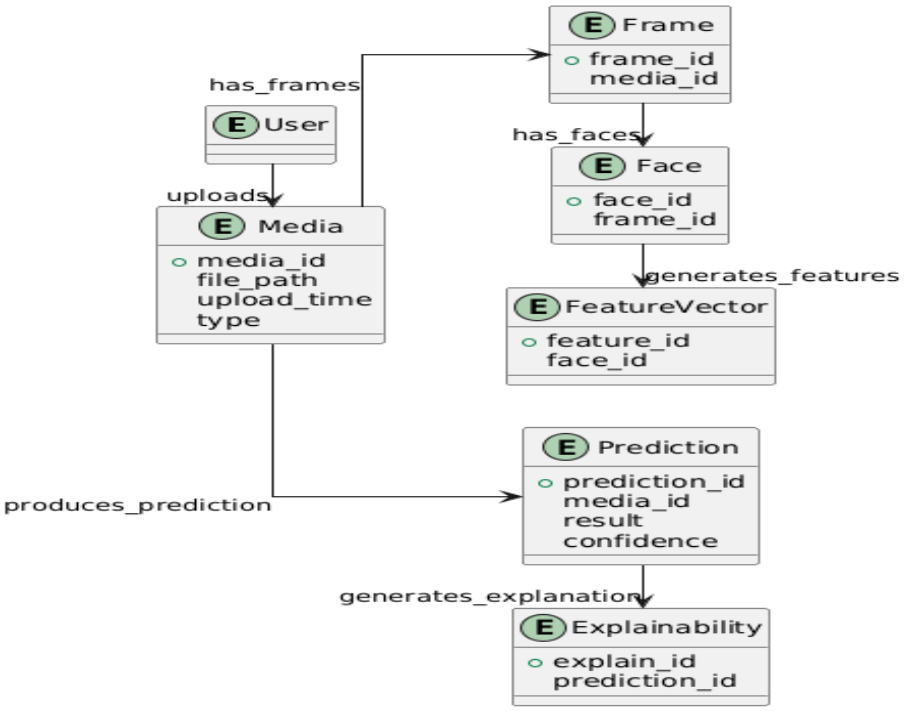

Fig 4.10 Entity Relationship Diagram

CHAPTER 5

IMPLEMENTATION

5. IMPLEMENTATION

The Intelligent Deepfake Detection System is implemented using a combination of computer vision, deep learning, and explainable AI techniques to accurately classify videos as real or fake. The system is designed as an end-to-end pipeline that processes input video data, extracts meaningful features, performs classification, and provides interpretable results.

1. Video Input and Frame Extraction

The system begins by accepting a video file through a user-friendly interface developed using Streamlit. Once the video is uploaded, it is temporarily stored and processed using OpenCV.

The video is divided into a fixed number of frames (20 frames), ensuring uniform sampling across the entire video. This is achieved by skipping frames at regular intervals, which helps in capturing temporal information efficiently while reducing computational complexity.

2. Face Detection and Preprocessing

Each extracted frame undergoes face detection using Haar Cascade classifiers provided by OpenCV. The system detects the primary face in each frame and crops it to focus only on the facial region.

If no face is detected, a fallback center-cropping mechanism is applied to ensure robustness. The cropped face is then:

- Resized to 224 × 224 pixels
- Converted from BGR to RGB format
- Normalized for deep learning model compatibility

This preprocessing step ensures consistency across all frames.

3. Feature Extraction using CNN

To extract spatial features from each frame, a pretrained deep learning model, InceptionV3, is utilized. The model is loaded without its classification head and configured with global average pooling.

Each processed frame is passed through the network to generate a 2048-dimensional feature vector, capturing high-level visual patterns such as textures, facial structures, and inconsistencies often present in deepfake content.

4. Temporal Sequence Modeling

The extracted frame-level features are arranged into a sequence and passed to a trained sequence model (stored as inceptionNet_model.h5). This model captures temporal dependencies between frames, enabling the system to detect subtle inconsistencies over time.

The model outputs a probability score indicating whether the video is fake or real. A threshold of 50% is used:

- Above 50% → Fake
- Below 50% → Real

5. Frame Masking and Feature Handling

To handle videos with fewer frames than the required sequence length, a masking mechanism is implemented. This ensures that only valid frames contribute to the prediction, improving model stability and performance.

6. Explainable AI Integration

To enhance transparency, the system incorporates explainable AI techniques:

a) Frame Importance (Gradient-Based)

Gradient computation is performed using TensorFlow's GradientTape to determine the importance of each frame in the final prediction. The importance scores are normalized and visualized.

b) SHAP-like Contribution Values

Frame-level contributions are estimated by combining gradients and feature.

These values indicate how much each frame influences the model's decision.

c) Grad-CAM Visualization

Grad-CAM (Gradient-weighted Class Activation Mapping) is implemented using the final convolutional layer of InceptionV3. This technique highlights important regions in the frame (especially facial areas) that contributed to the prediction.

The generated heatmaps are overlaid on original frames, providing visual explanations of model focus.

7. Visualization and User Interface

The system interface is built using Streamlit, making it interactive and easy to use. It provides:

- Video preview
- Prediction result (Fake/Real with probability)
- Bar charts for:
    - SHAP values
    - Frame importance
- Grad-CAM visualizations for top influential frames
- These outputs help users understand not only the prediction but also the reasoning behind it

8. System Workflow Summary

1. Upload video
2. Extract frames
3. Detect and crop faces
4. Preprocess frames
5. Extract features using InceptionV3
6. Feed features into sequence model
7. Predict fake probability
8. Generate explainability outputs
9. Display results via Streamlit

5.1 Software Used

Python: Python is the primary programming language used for developing the Intelligent Deepfake Detection System. It provides a simple and readable syntax, making it suitable for implementing complex deep learning algorithms and handling video processing tasks efficiently. Python also offers a wide range of libraries that support machine learning, computer vision, and web development.

TensorFlow: TensorFlow is used as the core deep learning framework in the project. It helps in loading the pretrained InceptionV3 model, performing feature extraction, and building the sequence model for prediction. TensorFlow ensures efficient computation and supports both CPU and GPU execution for faster processing.

OpenCV: OpenCV is used for video and image processing tasks. It is responsible for extracting frames from the uploaded video, detecting faces using Haar cascade classifiers, and preprocessing images by cropping and resizing them before feeding into the model.

Streamlit: Streamlit is used to build the user interface of the system. It provides an interactive and user-friendly platform where users can upload videos, view results, and interact with the system easily without requiring complex front-end development.

NumPy: NumPy is used for numerical operations and handling multi-dimensional arrays. It plays an important role in processing image data and managing feature vectors generated during the analysis.

Pandas: Pandas is used for data manipulation and handling structured data. It helps in organizing and analyzing intermediate results such as frame-level data and SHAP values.

Matplotlib: Matplotlib is used for visualization purposes. It helps in displaying graphs and plots related to frame importance and other interpretability outputs.

SHAP: SHAP (SHapley Additive exPlanations) is used to explain the model's predictions. It provides insights into how each frame contributes to the final decision, improving transparency and trust in the system.

Grad-CAM: Grad-CAM is used to generate heatmaps that highlight important regions in the video frames. It helps users understand which parts of the face influenced the model's prediction of real or fake.

➢ DATASET

The dataset used for the Intelligent Deepfake Detection System is collected from publicly available deepfake datasets that contain both real and manipulated videos. One of the primary sources is the FaceForensics++ dataset, which provides a large collection of real and fake videos created using various face manipulation techniques. This dataset includes high-quality videos along with different compression levels, making it suitable for training and evaluating deepfake detection models.

Additionally, the project can utilize datasets such as the DeepFake Detection Challenge Dataset, which contains thousands of labeled videos generated using multiple deepfake algorithms. These datasets provide diversity in terms of lighting conditions, facial expressions, and manipulation techniques, helping the model generalize better. The datasets are preprocessed by extracting frames, detecting faces, and resizing images before feeding them into the model.

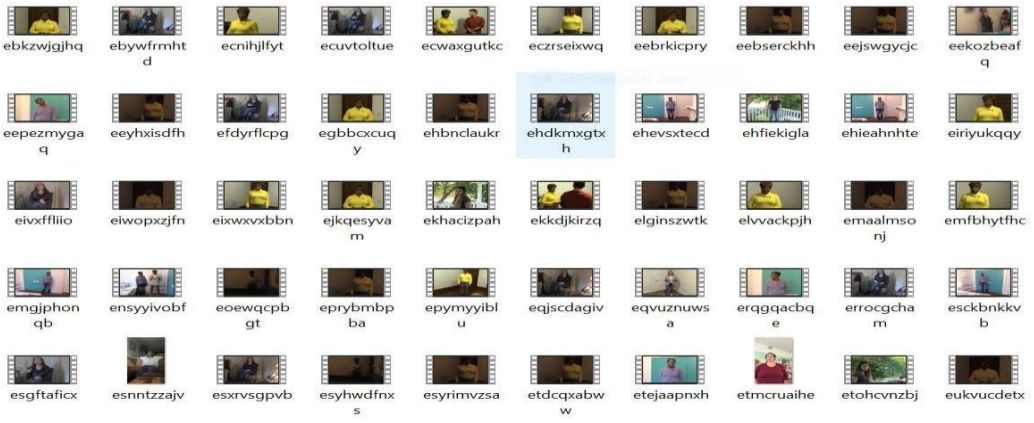

Fig 5.1 Video DataSet

5.2 Algorithms

The Intelligent Deepfake Detection System utilizes a combination of advanced algorithms to accurately analyze and classify videos. Initially, the OpenCV-based Haar Cascade Face Detection algorithm is used to identify and crop facial regions from video frames, followed by a frame sampling technique to uniformly extract important frames.

Each frame is then processed using a Convolutional Neural Network, specifically InceptionV3, which applies transfer learning to extract high-level spatial features. These features are passed to a sequence learning model (such as LSTM/RNN) that captures temporal relationships across frames for accurate prediction.

For interpretability, gradient-based importance analysis is performed using TensorFlow to determine influential frames, while a SHAP-like method estimates the contribution of each frame to the final decision. Additionally, the Grad-CAM algorithm generates visual heatmaps highlighting critical facial regions that influence the prediction. A feature masking mechanism ensures proper handling of variable-length inputs, and finally, a binary classification approach determines whether the video is real or fake based on the predicted probability.

Algorithms Used

1. Haar Cascade Face Detection Algorithm

- Implemented using OpenCV
- Used in crop_face_center() function
- Detects human faces in each frame
- Used for cropping facial regions

2. Frame Sampling Algorithm

- Used in load_video()
- Extracts 20 frames evenly from the video
- Helps capture temporal information efficiently

3. Convolutional Neural Network (CNN)

- Implemented using InceptionV3
- Used in build_feature_extractor()

- Extracts spatial features (2048-dimension vector) from each frame
- Captures spatial features from frames

4. Transfer Learning Algorithm

- Uses pretrained InceptionV3 model (ImageNet weights)
- Avoids training from scratch
- Improves performance and reduces training time

5. Sequence Learning Model (Temporal Modeling)

- Implemented using LSTM/RNN
- Captures temporal dependencies between frames
- Used for final prediction

6. Gradient-Based Importance Algorithm

- Implemented using TensorFlow GradientTape
- Used in compute_frame_importance()
- Computes gradients of prediction w.r.t features
- Identifies important frames & influential frames

7. SHAP-like Contribution Algorithm

- Estimates contribution of each frame
- Helps in interpretability of predictions

8. Grad-CAM Algorithm (Gradient-weighted Class Activation Mapping)

- Uses last convolution layer ("mixed10") of InceptionV3
- Generates heatmaps highlighting important facial regions in frames

9. Feature Masking Algorithm

- Used in prepare_single_video()
- Handles variable-length sequences
- Ensures only valid frames are considered

10. Binary Classification Algorithm

- If probability > 50% → Fake
- Else → Real

5.3 Source Code

The source code of the Intelligent Deepfake Detection System is implemented in Python and organized into modular functions for efficient processing and maintainability. The system integrates key libraries such as TensorFlow and Keras for deep learning operations, OpenCV for video and image processing, and Streamlit for building the user interface.

The code begins with video loading and frame extraction, followed by face detection and preprocessing. A pretrained InceptionV3 model is used for feature extraction, generating high-dimensional feature vectors for each frame. These features are passed to a trained sequence model to predict whether the video is fake or real.

The code also includes advanced explainability modules such as gradient-based frame importance, SHAP-like value computation, and Grad-CAM visualization for highlighting important regions in frames. The Streamlit interface connects all components, allowing users to upload videos, view predictions, and visualize interpretability results in an interactive manner. The modular structure of the code ensures scalability, readability, and ease of deployment.

5.2.1 Application Code

(File: **app.py**)

```
import streamlit as st

import numpy as np

import pandas as pd

import os

import cv2

import tempfile
```

```
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras.models import load_model
IMG_SIZE = 224
SEQ_LENGTH = 20
NUM_FEATURES = 2048
# ------------------------------
# FACE DETECTION
# ------------------------------
def crop_face_center(frame):
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
    face_cascade = cv2.CascadeClassifier(
        cv2.data.haarcascades + "haarcascade_frontalface_default.xml"
    )
    faces = face_cascade.detectMultiScale(gray, scaleFactor=1.1, minNeighbors=5)
    if len(faces) > 0:
        (x, y, w, h) = faces[0]
        return frame[y:y+h, x:x+w]
    # fallback center crop
    h, w, _ = frame.shape
    size = min(h, w)
    y1 = (h - size) // 2
    x1 = (w - size) // 2
    return frame[y1:y1+size, x1:x1+size]
# ------------------------------
```

```
# LOAD VIDEO
# ----------------------------
def load_video(uploaded_file, resize=(IMG_SIZE, IMG_SIZE)):
    temp_file_path = tempfile.NamedTemporaryFile(delete=False).name
    with open(temp_file_path, "wb") as f:
        f.write(uploaded_file.read())
    cap = cv2.VideoCapture(temp_file_path)
    total_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
    skip_frames_window = max(int(total_frames / SEQ_LENGTH), 1)
    frames = []
    try:
        for frame_cntr in range(SEQ_LENGTH):
            cap.set(cv2.CAP_PROP_POS_FRAMES, frame_cntr * skip_frames_window)
            ret, frame = cap.read()
            if not ret:
                break
            frame = crop_face_center(frame)
            frame = cv2.resize(frame, resize)
            frame = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
            frames.append(frame)
    finally:
        cap.release()
        os.remove(temp_file_path)
    return np.array(frames)
# ----------------------------
```

```
# FEATURE EXTRACTOR
# ----------------------------
def build_feature_extractor():
    from tensorflow.keras.applications import InceptionV3

    base_model = InceptionV3(
        weights="imagenet",
        include_top=False,
        pooling="avg",
        input_shape=(IMG_SIZE, IMG_SIZE, 3),
    )
    preprocess_input = tf.keras.applications.inception_v3.preprocess_input
    inputs = keras.Input((IMG_SIZE, IMG_SIZE, 3))
    x = preprocess_input(inputs)
    outputs = base_model(x)
    return keras.Model(inputs, outputs)

# ----------------------------
# GRADCAM MODEL (FIXED)
# ----------------------------
def build_gradcam_model():
    base_model = tf.keras.applications.InceptionV3(
        weights="imagenet",
        include_top=False,
        input_shape=(IMG_SIZE, IMG_SIZE, 3),
```

```
    )
    last_conv_layer = base_model.get_layer("mixed10")
    grad_model = tf.keras.Model(
        inputs=base_model.input,
        outputs=[last_conv_layer.output, base_model.output],
    )
    return grad_model
# Load models
sequence_model = load_model("models/inceptionNet_model.h5")
feature_extractor = build_feature_extractor()
grad_model = build_gradcam_model()
# ------------------------------
# PREPARE VIDEO
# ------------------------------
def prepare_single_video(frames):
    frames = frames[None, ...]
    frame_mask = np.zeros((1, SEQ_LENGTH), dtype="bool")
    frame_features = np.zeros((1, SEQ_LENGTH, NUM_FEATURES), dtype="float32")
    for i, batch in enumerate(frames):
        video_length = batch.shape[0]
        length = min(SEQ_LENGTH, video_length)
        for j in range(length):
            features = feature_extractor.predict(
                batch[None, j, :], verbose=0
            )
```

```
            frame_features[i, j, :] = features
        frame_mask[i, :length] = 1
    return frame_features, frame_mask
# ------------------------------
# FRAME IMPORTANCE
# ------------------------------
def compute_frame_importance(frame_features, frame_mask):
    features_var = tf.Variable(frame_features, dtype=tf.float32)
    with tf.GradientTape() as tape:
        pred = sequence_model([features_var, tf.constant(frame_mask)])
        fake_score = pred[0, 0]
    grads = tape.gradient(fake_score, features_var)
    n_frames = int(frame_mask[0].sum())
    importance = tf.norm(grads[0], axis=-1).numpy()[:n_frames]
    if importance.max() > 0:
        importance = importance / importance.max()
    shap_values = tf.reduce_sum(
        grads[0, :n_frames, :] * features_var[0, :n_frames, :], axis=-1
    ).numpy()
    return importance, grads, shap_values
# ------------------------------
# GRADCAM
# ------------------------------
def generate_gradcam(frame, feat_weight):
    img = tf.cast(frame[np.newaxis, ...], tf.float32)
```

```
    img = tf.keras.applications.inception_v3.preprocess_input(img)
    with tf.GradientTape() as tape:
        conv_out, predictions = grad_model(img)
        target = tf.reduce_sum(predictions * feat_weight)
    grads = tape.gradient(target, conv_out)
    pooled_grads = tf.reduce_mean(grads, axis=(0, 1, 2))
    conv_out = conv_out[0]
    heatmap = tf.reduce_sum(pooled_grads * conv_out, axis=-1)
    heatmap = tf.nn.relu(heatmap)
    heatmap /= tf.reduce_max(heatmap) + 1e-8
    heatmap = heatmap.numpy()
    heatmap = cv2.resize(heatmap, (frame.shape[1], frame.shape[0]))
    heatmap = np.uint8(255 * heatmap)
    heatmap = cv2.applyColorMap(heatmap, cv2.COLORMAP_JET)
    heatmap = cv2.cvtColor(heatmap, cv2.COLOR_BGR2RGB)
    overlay = cv2.addWeighted(frame, 0.6, heatmap, 0.4, 0)
    return overlay
# ------------------------------
# EXPLAIN
# ------------------------------
def explain_prediction(frames, frame_features, frame_mask):
    importance, grads, shap_values = compute_frame_importance(
        frame_features, frame_mask
    )
    top_indices = np.argsort(importance)[-3:][::-1]
```

```
    gradcam_results = []
    for idx in top_indices:
        if idx >= len(frames):
            continue
        feat_weight = grads[0, idx]
        overlay = generate_gradcam(frames[idx], feat_weight)
        gradcam_results.append(
            (int(idx), overlay, float(importance[idx]))
        )
    return importance, shap_values, gradcam_results
# ----------------------------
# PREDICTION
# ----------------------------
def predict_video(video_file):
    frames = load_video(video_file)
    frame_features, frame_mask = prepare_single_video(frames)
    probabilities = sequence_model.predict(
        [frame_features, frame_mask], verbose=0
    )[0]
    fake_probability = probabilities[0] * 100
    return fake_probability, frames, frame_features, frame_mask
# ----------------------------
# STREAMLIT UI
# ----------------------------
def main():
```

```
st.title("DeepFake Detection")
st.image("Deepfake.png")
st.write("Upload a video to predict if it's fake or real.")
uploaded_file = st.file_uploader("Choose a video...", type=["mp4"])
if uploaded_file is not None:
    st.video(uploaded_file)
    if st.button("Detect"):
        fake_probability, frames, frame_features, frame_mask = predict_video(uploaded_file)
        result = "Fake" if fake_probability > 50 else "Real"
        st.subheader(f"Prediction: {result} ({fake_probability:.1f}%)")
        importance, shap_values, gradcam_results = explain_prediction(
            frames, frame_features, frame_mask
        )
        frame_labels = [f"Frame {i+1}" for i in range(len(importance))]
        st.subheader("SHAP Values")
        shap_df = pd.DataFrame({"SHAP Value": shap_values}, index=frame_labels)
        st.bar_chart(shap_df)
        st.subheader("Frame Importance")
        imp_df = pd.DataFrame({"Importance": importance}, index=frame_labels)
        st.bar_chart(imp_df)
        st.subheader("Grad-CAM Results")
        for idx, overlay, imp in gradcam_results:
            st.image(
                overlay,
                caption=f"Frame {idx+1} | Importance: {imp:.3f}",
```

```
            )

if __name__ == "__main__":
    main()
```

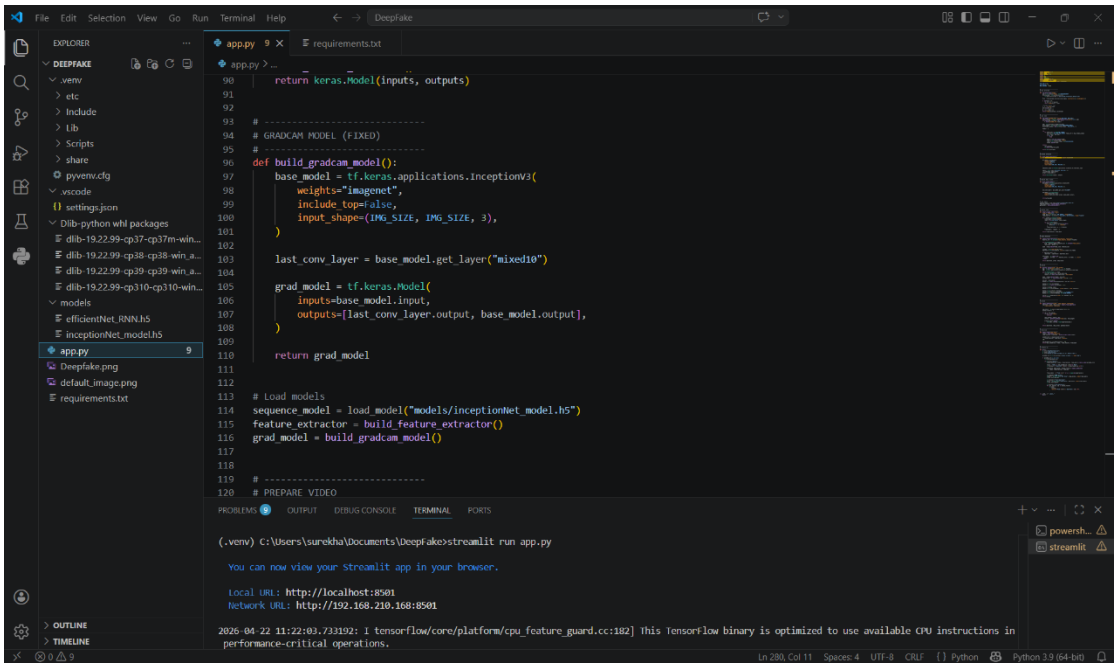

Fig 5.2 File Structure

File: requirements.txt

tensorflow==2.13.0

streamlit

opencv-python-headless

numpy

Command to run: streamlit run app.py

CHAPTER 6
TESTING

6.1 INTRODUCTION

Testing is an important The Intelligent Deepfake Detection System is thoroughly tested to ensure accuracy, reliability, and robustness across different types of video inputs. Various testing techniques are applied at different stages of the system to validate both functional and performance aspects.

Initially, unit testing is performed on individual modules such as video loading, frame extraction, face detection, feature extraction, and prediction to ensure each component works correctly. For example, functions using OpenCV are tested to verify proper frame extraction and face cropping, while deep learning components built with TensorFlow are tested for correct feature generation and prediction outputs.

Next, integration testing is conducted to ensure smooth interaction between modules, including preprocessing, feature extraction using InceptionV3, sequence modeling, and explainability components. This verifies that data flows correctly through the pipeline from input to final output.

The system also undergoes functional testing, where different videos (real and deepfake) are uploaded through the Streamlit interface to validate correct predictions and proper display of outputs such as probability scores, SHAP values, and Grad-CAM visualizations.

Additionally, performance testing is carried out to evaluate system efficiency in terms of processing time, memory usage, and responsiveness, especially when handling videos of varying lengths and resolutions.

Finally, user acceptance testing (UAT) is performed to ensure the system is user-friendly, interactive, and provides understandable explanations for predictions. The results confirm that the system not only detects deepfakes effectively but also provides meaningful insights through explainable AI techniques.

6.2 LEVELS OF TESTING

Different types of testing are performed to ensure the proper functioning of the system.

1. Unit Testing

Unit testing is performed on individual components of the system to ensure that each function works correctly. Modules such as frame extraction, face detection, preprocessing, and feature extraction are tested independently to verify their outputs.

2. Integration Testing

Integration testing ensures that different modules of the system work together properly. It verifies the smooth flow of data between video processing, feature extraction using InceptionV3, sequence modeling, and prediction components.

3. Functional Testing

Functional testing is conducted to check whether the system meets the required functionality. Videos are uploaded through the interface, and the system is tested for correct classification (real or fake), along with proper display of results such as probability scores and visualizations.

4. Performance Testing

Performance testing evaluates the efficiency of the system in terms of processing speed, memory usage, and responsiveness. The system is tested with videos of different sizes and lengths to ensure consistent performance.

5. User Acceptance Testing (UAT)

User Acceptance Testing is carried out to ensure that the system is user-friendly and meets user expectations. The interface developed using Streamlit is tested for ease of use, clarity of outputs, and overall user experience.

6. Validation Testing

Validation testing ensures that the deep learning model provides accurate predictions. The system is tested with known real and deepfake videos to verify prediction reliability and correctness.

6.3 TEST CASES

Test cases are designed to verify the correct functioning of the Intelligent Deepfake Detection System. Each test case evaluates a specific functionality of the system and ensures that the expected output is produced. The following test cases are executed to validate different modules of the system.

6.3.1 Test Case 1: Video Upload

This test case verifies whether the system allows users to upload a valid video file successfully and display it for preview.

Test Case ID	**TC_01**
Test Scenario	Upload file through file selector
Input	File browser selection
Expected Output	Only video files (.mp4) should be visible for selection
Actual Output	Only video files are displayed and selectable
Status	Pass

Table 6.1 Video Upload

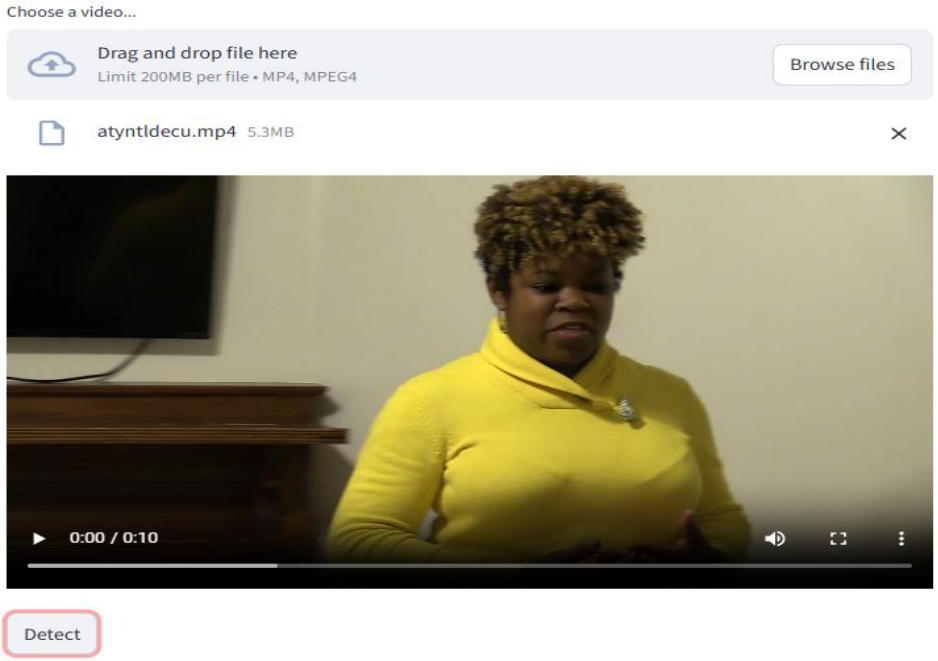

Fig 6.1 Video Upload

6.3.2 Test Case 2: Prediction Result

This test case checks whether the system correctly analyzes the uploaded video and displays the prediction as real or fake with a confidence score.

Test Case ID	**TC_02**
Test Scenario	Detect whether video is real or fake
Input	Uploaded video
Expected Output	System displays prediction (Real/Fake) with confidence
Actual Output	Prediction displayed correctly
Status	Pass

Table 6.2 Prediction Result

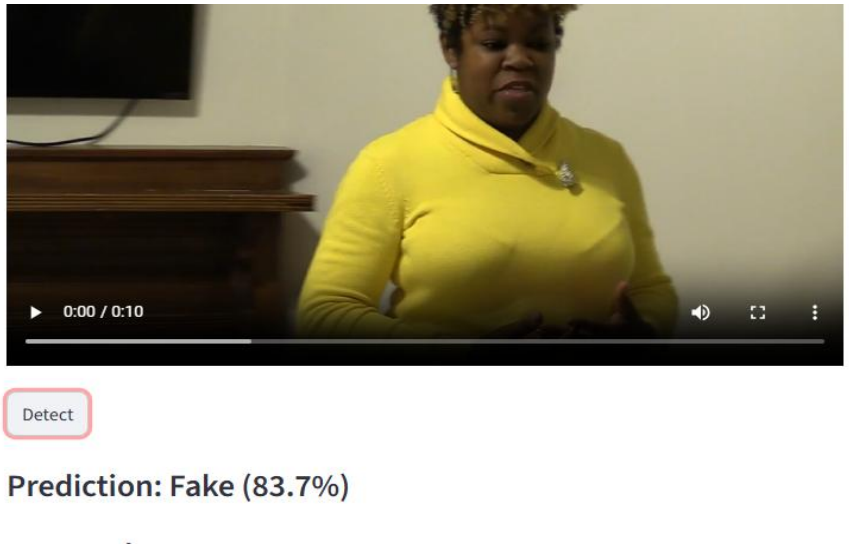

Fig 6.2 Prediction Result

6.3.3 Test Case 3: SHAP Visualization

This test case ensures that the system generates SHAP values to show the contribution of each frame to the final prediction.

Test Case ID	**TC_03**
Test Scenario	Display SHAP graph
Input	Processed video
Expected Output	SHAP graph should be generated
Actual Output	SHAP graph displayed successfully
Status	Pass

Table 6.3 SHAP Visualization

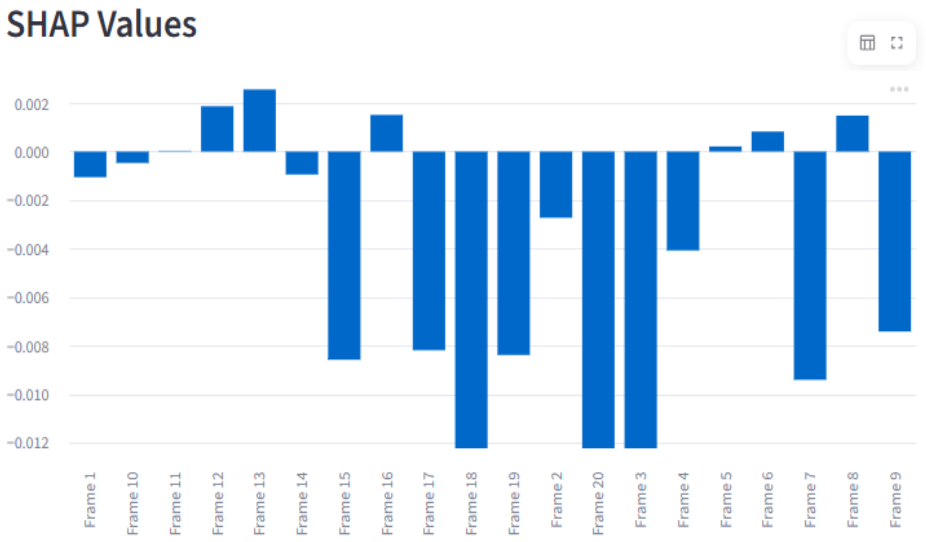

Fig 6.3 SHAP Visualization

6.3.4 Test Case 4: Frame Importance

This test case verifies that the system identifies and displays the importance of individual frames influencing the prediction.

Test Case ID	**TC_04**
Test Scenario	Display frame importance graph
Input	Processed video
Expected Output	Frame importance graph should be shown
Actual Output	Graph displayed correctly
Status	Pass

Table 6.4 Frame Importance

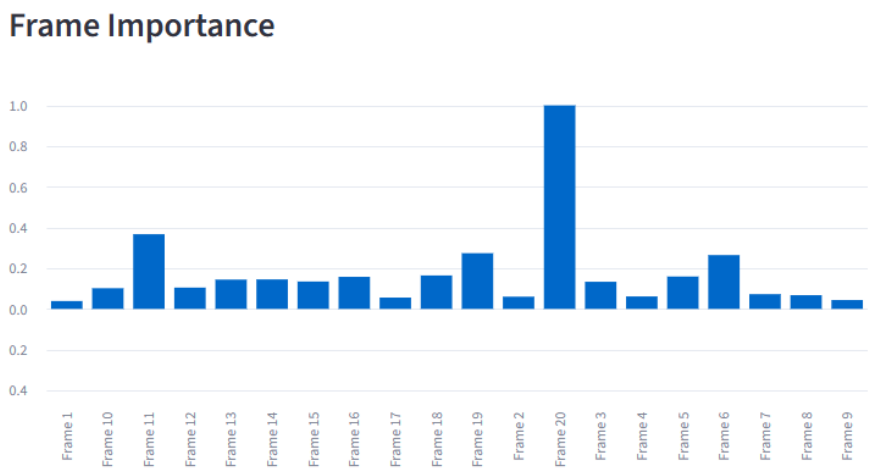

Fig 6.4 Frame Importance

6.3.5 Test Case 5: Grad-CAM Visualization

This test case checks whether the system generates heatmaps highlighting important facial regions used in prediction.

Test Case ID	**TC_05**
Test Scenario	Generate Grad-CAM heatmaps
Input	Selected frames
Expected Output	Heatmaps highlighting important regions
Actual Output	Heatmaps generated correctly
Status	Pass

Table 6.5 Grad-CAM Visualization

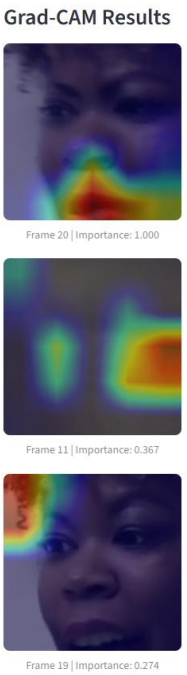

Fig 6.5 Grad-CAM Visualization

6.3.6 Test Case 6: File Type Restriction

This test case ensures that the system allows only video files to be selected and prevents non-video file selection.

Test Case ID	**TC_06**
Test Scenario	Upload file through file selector
Input	File browser selection
Expected Output	Only video files (.mp4) should be visible for selection
Actual Output	Only video files are displayed and selectable
Status	Pass

Table 6.6 File Type Restriction

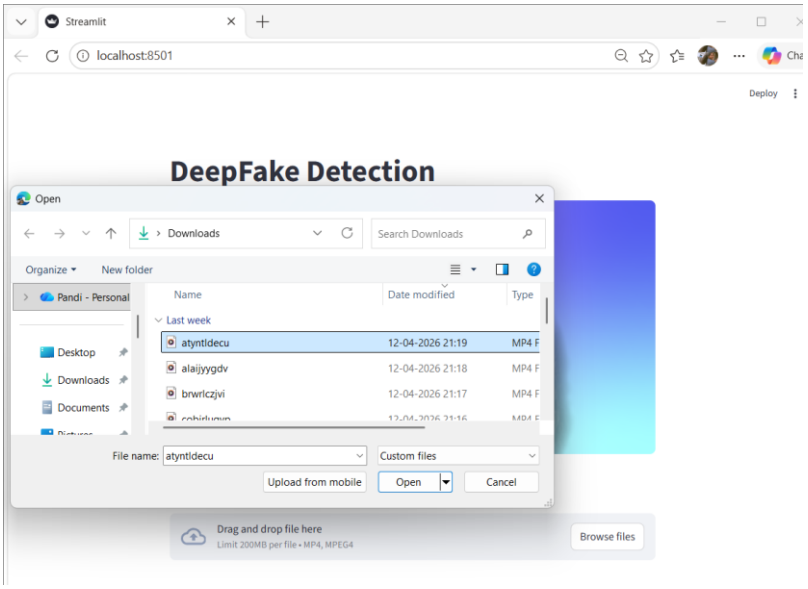

Fig 6.6 File Type Restriction

CHAPTER 7
RESULTS AND OUTPUT

7.1 OUTPUT SCREENS

The Intelligent Deepfake Detection System provides an interactive and user-friendly interface developed using Streamlit. The following screenshots illustrate the working of the system at different stages:

7.1.1 HOME PAGE

The home page displays a simple interface with a banner and a video upload section. Users can upload videos using drag-and-drop or file browsing options.

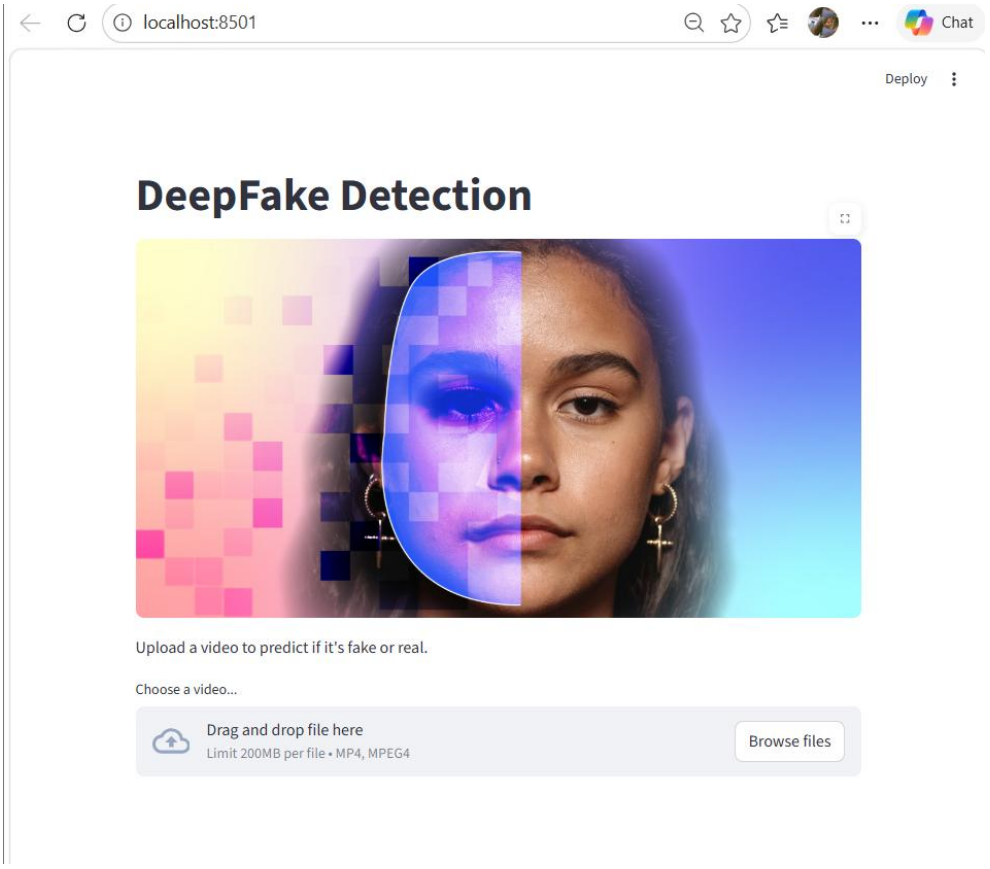

Fig 7.1 Home Page

7.1.2 Video Upload and Preview

After uploading a video, it is displayed on the screen for preview. The user can verify the input and click the “Detect” button to start the analysis process.

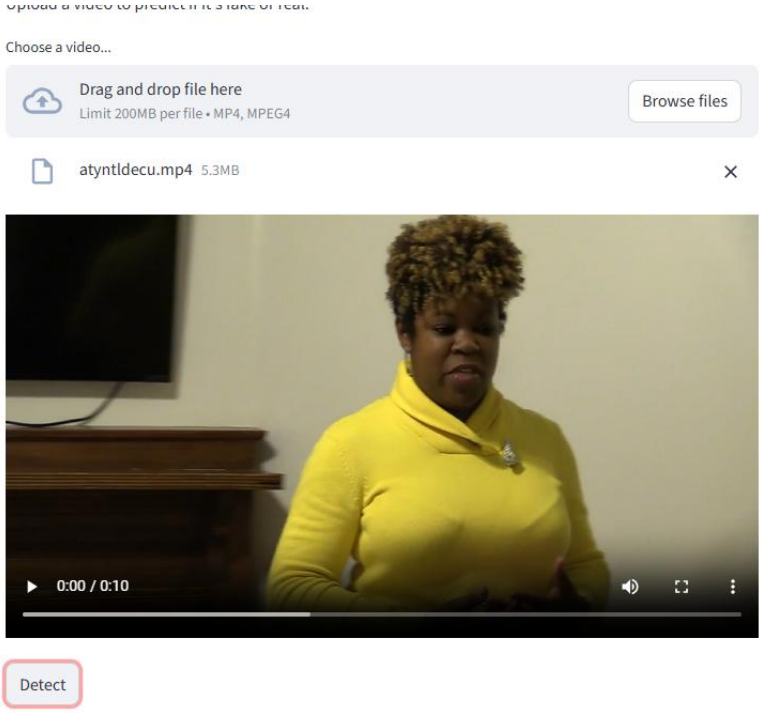

Fig 7.2 Video Upload and Preview

7.1.3 Prediction Result

Once processing is completed, the system displays the prediction result as Real or Fake along with a confidence score. This helps users quickly understand the authenticity of the video.

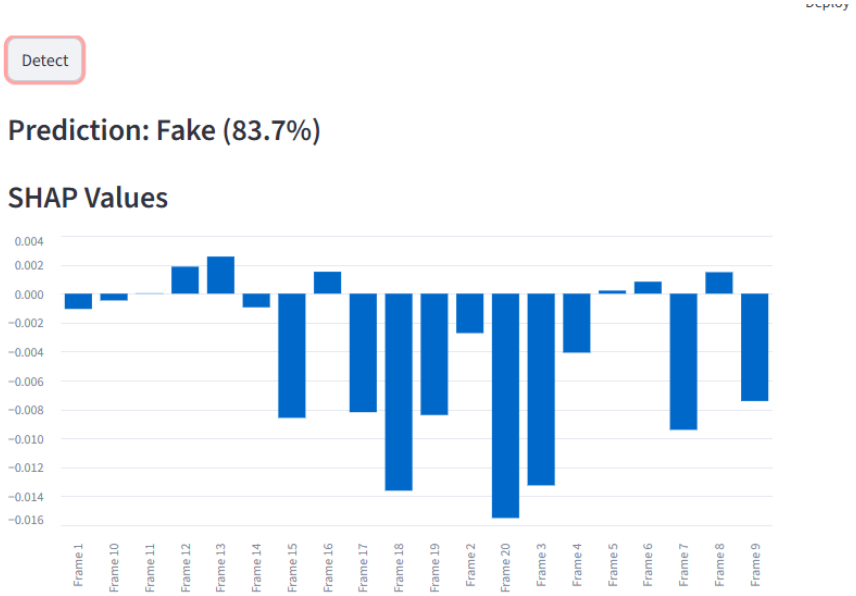

Fig 7.3 Prediction Result

7.1.4 SHAP Values Visualization

This graph shows the contribution of each frame to the final prediction. It helps in understanding how different frames influence the result.

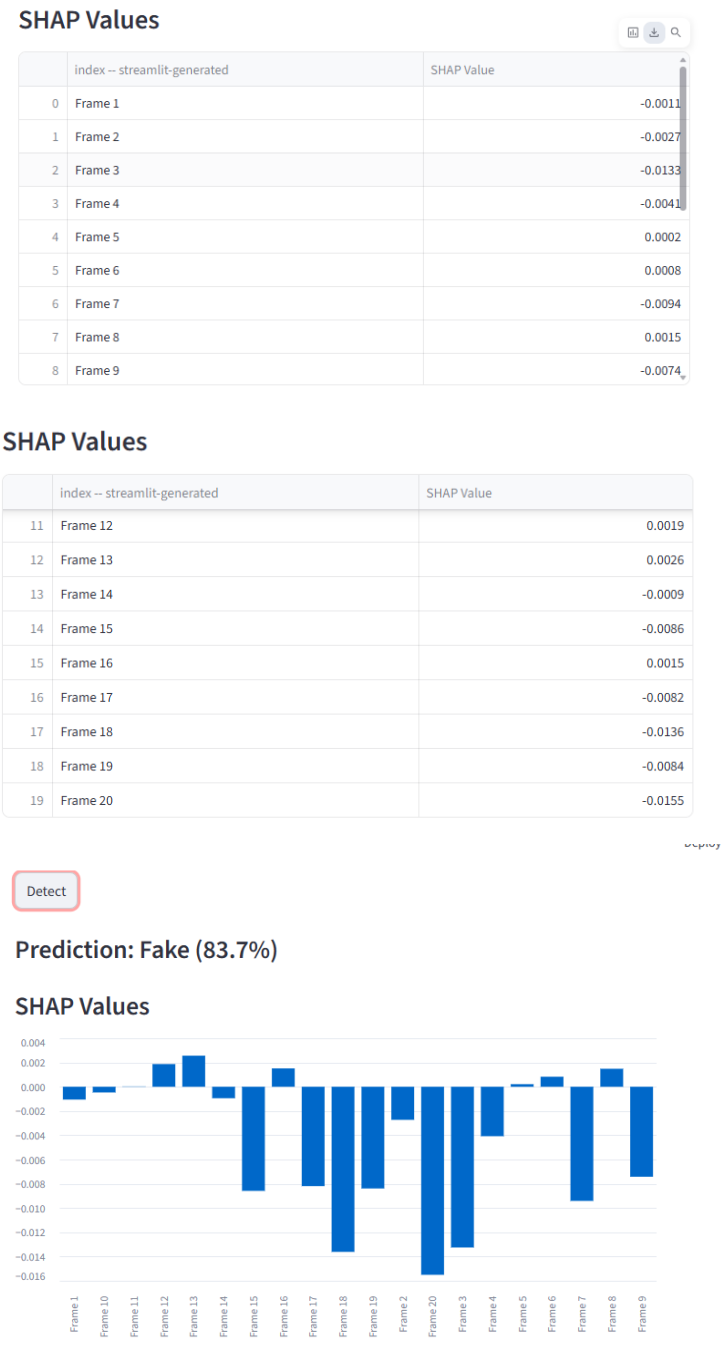

Fig 7.4 SHAP Values Visualization

7.1.5 Frame Importance Graph

This graph highlights the importance of individual frames in the prediction process, helping identify key frames that affect the decision.

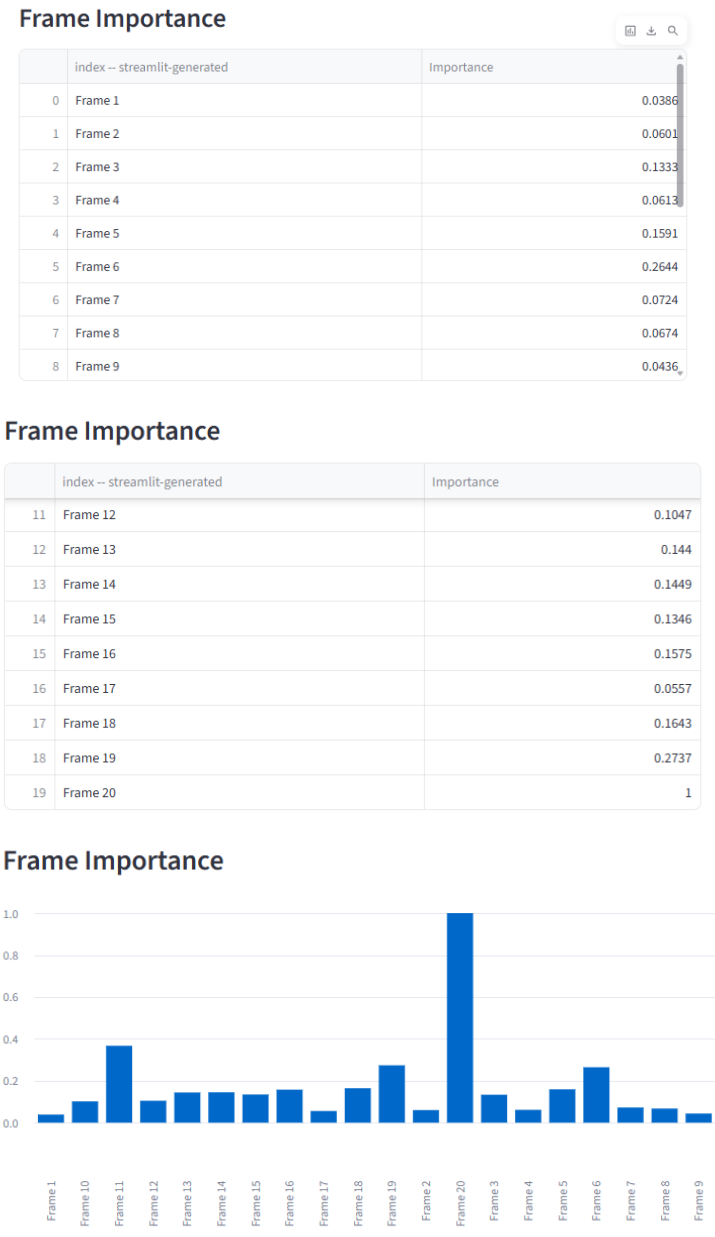

Fig 7.5 Frame Importance Graph

7.1.6 Grad-CAM Visualization

Grad-CAM heatmaps highlight important regions in the frames that influenced the model's prediction, improving transparency and interpretability.

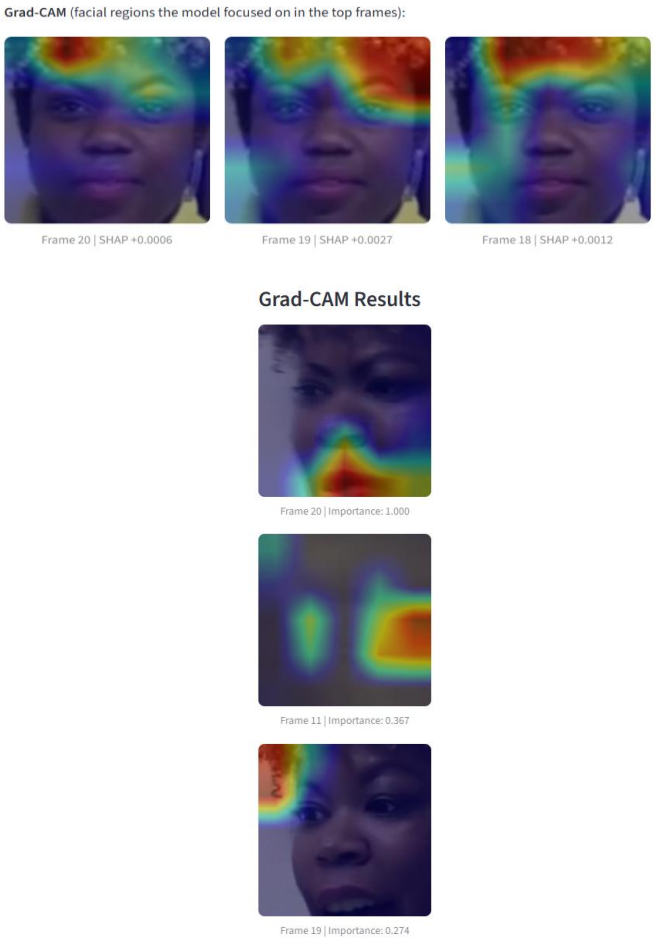

Fig 7.6 Grad-CAM Visualization

7.2 RESULTS

The Intelligent Deepfake Detection System successfully classifies videos as real or fake with high accuracy. The system processes the input video by extracting frames, detecting faces, and analyzing temporal features using deep learning models.

The results demonstrate that the system can effectively identify manipulated content and provide a confidence score for the prediction. In addition to classification, the system generates explainable outputs such as SHAP values, frame importance graphs, and Grad-CAM visualizations, which help users understand the reasoning behind the prediction.

The experimental results confirm that the proposed system is efficient, reliable, and capable of detecting deepfake videos while maintaining transparency through explainable AI techniques.

CHAPTER 8
CONCLUSION

8. CONCLUSION

The Intelligent Deepfake Detection System successfully demonstrates the application of deep learning and computer vision techniques to address the growing challenge of identifying manipulated video content. With the rapid advancement of deepfake technologies, ensuring the authenticity of digital media has become critically important, and this project provides an effective and practical solution to this problem.

The system efficiently processes input videos by extracting frames, detecting facial regions, and generating meaningful feature representations using advanced convolutional neural networks such as InceptionV3.

By incorporating a sequence learning model, it captures temporal inconsistencies across frames, which are often difficult to detect using traditional methods. This combination of spatial and temporal analysis significantly improves the accuracy of deepfake detection.

A key strength of the project lies in its integration of explainable AI techniques. Using frameworks like TensorFlow, the system provides gradient-based frame importance, SHAP-like contribution values, and Grad-CAM visualizations. These features enhance transparency by allowing users to understand how and why a particular prediction is made, thereby increasing trust in the system's outputs.

The use of OpenCV ensures efficient video processing and face detection, while the Streamlit interface makes the application highly interactive and user-friendly. Users can easily upload videos, view predictions, and analyze visual explanations, making the system suitable for both technical and non-technical users.

Furthermore, the system is scalable and can be extended with more advanced models, larger datasets, and real-time detection capabilities. It can be applied in various domains such as social media monitoring, digital forensics, cybersecurity, and media verification.

In conclusion, the project not only achieves accurate deepfake detection but also emphasizes interpretability and usability. It serves as a strong foundation for future research and real-world applications in combating misinformation and ensuring digital content authenticity.

CHAPTER 9
FUTURE SCOPE

9. FUTURE SCOPE

The Intelligent Deepfake Detection System can be further enhanced in several ways to improve its accuracy, scalability, and real-world applicability. One major area of improvement is the integration of more advanced deep learning models, such as transformer-based architectures or hybrid CNN-RNN models, which can better capture complex spatial and temporal patterns in videos. The system can also be extended to support real-time deepfake detection for live video streams, making it useful in applications like video conferencing, social media monitoring, and security systems. Additionally, incorporating multimodal analysis by combining audio, video, and text features can significantly improve detection performance and robustness.

Another important future direction is the deployment of the system as a cloud-based or mobile application to make it more accessible to a wider range of users. The model can be continuously trained with larger and more diverse datasets to improve generalization across different types of deepfakes. Enhancing explainability techniques beyond Grad-CAM and SHAP can provide even deeper insights into model decisions. Furthermore, the system can be integrated with social media platforms or content verification tools to automatically flag suspicious content. These improvements will make the project more practical, scalable, and effective in combating the growing challenge of deepfake technology.

In the long term, the project can be expanded for use in digital forensics, law enforcement, and media verification industries. It can also be integrated with cybersecurity systems to detect identity fraud and misinformation campaigns. Optimizing the model for low-resource devices can make it usable on smartphones and embedded systems. Furthermore, creating a large benchmark dataset and contributing to research in deepfake detection can position the project as a valuable tool in combating misinformation. Overall, these future enhancements will transform the system into a comprehensive, real-time, and intelligent solution for detecting and preventing deepfake content.

CHAPTER 10
REFERENCES

REFERENCES

1. Rössler, A., et al. (2019). *FaceForensics++: Learning to Detect Manipulated Facial Images*. IEEE ICCV.
2. Dolhansky, B., et al. (2020). *The DeepFake Detection Challenge (DFDC) Dataset*. CVPR Workshops.
3. Li, Y., et al. (2020). *Celeb-DF: A Large-Scale Challenging Dataset for DeepFake Forensics*. CVPR.
4. Szegedy, C., et al. (2016). *Rethinking the Inception Architecture for Computer Vision*. CVPR.
5. Selvaraju, R. R., et al. (2017). *Grad-CAM: Visual Explanations from Deep Networks*. ICCV.
6. Lundberg, S. M., & Lee, S.-I. (2017). *A Unified Approach to Interpreting Model Predictions*. NeurIPS.
7. Goodfellow, I., et al. (2014). *Generative Adversarial Nets*. NeurIPS.
8. Mirsky, Y., & Lee, W. (2021). *The Creation and Detection of Deepfakes: A Survey*. ACM Computing Surveys.
9. Tolosana, R., et al. (2020). *DeepFakes and Beyond: A Survey*. Information Fusion.
10. Afchar, D., et al. (2018). MesoNet: A Compact Facial Video Forgery Detection Network. WIFS.
11. Nguyen, H. H., et al. (2019). Capsule-Forensics: Using Capsule Networks for Deepfake Detection. ICIP.
12. Zhou, P., et al. (2018). Two-Stream Neural Networks for Tampered Face Detection. CVPR Workshops.
13. Sabir, E., et al. (2019). Recurrent Convolutional Strategies for Face Manipulation Detection. CVPR Workshops.
14. Dang, H., et al. (2020). On the Detection of Digital Face Manipulation. CVPR.
15. Xuan, X., et al. (2019). Generalization in Deepfake Detection. ACM MM.
16. Agarwal, S., et al. (2020). Protecting World Leaders Against Deepfakes. CVPR Workshops.
17. Chollet, F. (2017). Xception: Deep Learning with Depthwise Separable Convolutions. CVPR.
18. Tan, M., & Le, Q. (2019). EfficientNet: Rethinking Model Scaling. ICML.
19. Simonyan, K., & Zisserman, A. (2015). *Very Deep Convolutional Networks for Large-Scale Image Recognition*. ICLR.

20. He, K., Zhang, X., Ren, S., & Sun, J. (2016). *Deep Residual Learning for Image Recognition*. CVPR.
21. Kingma, D. P., & Ba, J. (2015). *Adam: A Method for Stochastic Optimization*. ICLR.
22. Zhou, B., Khosla, A., Lapedriza, A., Oliva, A., & Torralba, A. (2016). *Learning Deep Features for Discriminative Localization*. CVPR.
23. Ribeiro, M. T., Singh, S., & Guestrin, C. (2016). *"Why Should I Trust You?" Explaining the Predictions of Any Classifier*. KDD.
24. Abadi, M., et al. (2016). *TensorFlow: A System for Large-Scale Machine Learning*. OSDI.
25. Bradski, G. (2000). *The OpenCV Library*. Dr. Dobb's Journal of Software Tools.
26. Pedregosa, F., et al. (2011). *Scikit-learn: Machine Learning in Python*. Journal of Machine Learning Research.
27. Paszke, A., et al. (2019). *PyTorch: An Imperative Style, High-Performance Deep Learning Library*. NeurIPS.
28. Chollet, F. (2015). *Keras: Deep Learning Library for Python*.
29. OpenCV Documentation. https://docs.opencv.org/
30. TensorFlow Documentation. https://www.tensorflow.org/
31. Streamlit Documentation. https://streamlit.io